NOM DE VOTRE ÉTABLISSEMENT

Département de Génie Informatique

Rapport de travaux pratiques

Architecture et Systèmes (ASR2)

Du noyau Linux à l'espace utilisateur

LABO BETALAB

Travaux réalisés

01 **TP 01** – Compilation du noyau Linux et exploration du disque

02 **TP 02** – Programmation d'un mini-shell

03 **TP 03** – Threads POSIX et ordonnancement

04 **TP 04** – Implémentation d'un appel système

05 **TP 05** – Gestionnaire de mémoire dynamique (*heap manager*)

06 **TP 06** – Protection mémoire par pages de garde

07 **TP 07** – Suivi des pages *soft-dirty* d'un processus

08 **TP 08** – Module noyau et pilote en espace utilisateur (UIO)

09 **TP 09** – Communication entre processus : `pipe` et `dup2`

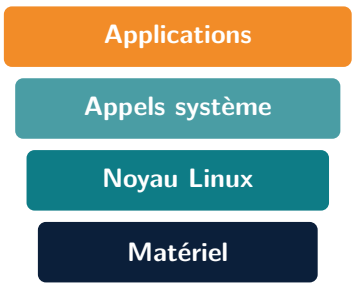
Réalisé par

LEA MEGNIJIO

23p403 | 4eme Annee Genie Informatique

Sous la supervision de

Professeur Alain Tchana



Année académique 2026 – 2027

Table des matières

Introduction générale **4**

Introduction générale **5**

0 Préparation de l'environnement de travail **6**
0.1 Vérification de la machine 6
0.2 Analyse et libération d'espace 7
0.3 Déplacement du Bureau vers la partition Windows 8
0.4 Difficultés rencontrées 8
0.5 Bilan 9

1 TP1 – Compilation du noyau Linux et exploration du disque **10**
1.1 Objectifs du TP 10
1.2 Notions clés 10
1.3 Adaptations par rapport au sujet 11
1.4 Partie 1 : compilation et exploration du noyau 11
1.4.1 Création de la machine virtuelle 11
1.4.2 Compilation 13
1.4.3 Installation et vérification 14
1.4.4 Extraction du binaire et recherche de `tcp_v4_do_rcv` 16
1.4.5 Cartographie de la mémoire physique 17
1.5 Partie 2 : exploration du disque 18
1.5.1 Ajout d'un disque de test 18
1.5.2 Partitionnement et formatage 19
1.5.3 Montage et fichier `/etc/fstab` 21
1.5.4 Sauvegarde, destruction et restauration du MBR avec `dd` 21
1.6 Difficultés rencontrées et solutions 24
1.7 Conclusion du TP 24

2 TP2 – Programmation d'un mini-shell **25**
2.1 Objectifs du TP 25
2.2 Notions clés 25
2.3 Déroulement – Partie 1 25
2.3.1 Étude du programme d'exemple `processus.c` 25
2.3.2 Boucle principale et lecture des commandes 26
2.3.3 Exécution des commandes externes et table des processus 27
2.3.4 Variables d'environnement : `export` et `env` 27
2.3.5 Recherche des commandes dans le `PATH` du shell 28
2.4 Déroulement – Partie 2 : signaux et gestion des processus 29

2.4.1 Ctrl-C et Ctrl-Z . . . 29
2.4.2 Arrière-plan et commandes internes . . . 30
2.5 Difficultés rencontrées et solutions . . . 31
2.6 Conclusion du TP . . . 31

3 TP3 – Threads POSIX et ordonnancement 32
3.1 Objectifs du TP . . . 32
3.2 Notions clés . . . 32
3.3 Déroulement . . . 32
3.3.1 Partie 1 : threads POSIX et signaux . . . 32
3.3.2 Partie notée : les contextes . . . 33
3.4 Conclusion du TP . . . 34

4 TP4 – Implémentation d'un appel système 35
4.1 Objectifs . . . 35
4.2 Notions clés . . . 35
4.3 Réalisation . . . 35
4.4 Difficultés et conclusion . . . 36

5 TP5 – Gestionnaire de mémoire dynamique (*heap manager*) 37
5.1 Objectifs . . . 37
5.2 Notions clés . . . 37
5.3 Réalisation . . . 37
5.4 Difficultés et conclusion . . . 39

6 TP6 – Protection mémoire par pages de garde 40
6.1 Objectifs . . . 40
6.2 Notions clés . . . 40
6.3 Réalisation . . . 40
6.4 Difficultés et conclusion . . . 41

7 TP7 – Suivi des pages *soft-dirty* d'un processus 42
7.1 Objectifs . . . 42
7.2 Notions clés . . . 42
7.3 Réalisation . . . 42
7.4 Difficultés et conclusion . . . 44

8 TP8 – Module noyau et pilote en espace utilisateur (UIO) 45
8.1 Objectifs . . . 45
8.2 Notions clés . . . 45
8.3 Réalisation . . . 45
8.4 Difficultés et conclusion . . . 47

9 TP9 – Communication entre processus : `pipe` et `dup2` 48
9.1 Objectifs . . . 48
9.2 Notions clés . . . 48
9.3 Réalisation . . . 48
9.4 Difficultés et conclusion . . . 49

Conclusion générale 50

Références **51**

A Codes sources **52**

A Codes sources **53**

Introduction générale

Introduction générale

Un système d'exploitation est souvent utilisé comme une boîte noire : on lance des programmes, on alloue de la mémoire, on enregistre des fichiers, sans voir les mécanismes qui le permettent. Les travaux pratiques du cours ASR2 (Architecture et Systèmes) ont pour but d'ouvrir cette boîte noire, en étudiant le fonctionnement interne de Linux, du noyau jusqu'aux programmes de l'espace utilisateur.

Les neuf TP suivent un parcours progressif :

- **le noyau** : compilation et installation d'un noyau Linux, et exploration du disque (TP1), ajout d'un appel système (TP4), écriture d'un module noyau et d'un pilote en espace utilisateur (TP8) ;
- **les processus et les threads** : réalisation d'un mini-shell (TP2), ordonnancement de threads par signaux et par contextes (TP3), communication entre processus par tube (TP9) ;
- **la mémoire** : écriture d'un gestionnaire de tas (TP5), sa protection contre les débordements de tampon (TP6), et le suivi des pages modifiées d'un processus (TP7).

Les TP ont été réalisés sur un ordinateur portable sous Ubuntu 24.04 (15 Go de RAM, 8 cœurs). Ceux qui modifient le noyau ont été menés dans une machine virtuelle VirtualBox sous Ubuntu Server 20.04, afin de ne pas mettre en danger la machine hôte ; les autres, écrits en C, ont été réalisés directement sur l'hôte. Le sujet ayant été conçu pour un environnement Xen, les adaptations nécessaires sont signalées dans chaque chapitre. Pour chaque TP, nous présentons les objectifs, les notions utilisées, le déroulement avec les résultats obtenus, puis les difficultés rencontrées. Les codes sources sont remis séparément.

Chapitre 0

Préparation de l'environnement de travail

Les TP 1, 4 et 8 nécessitent de compiler le noyau Linux dans une machine virtuelle, ce qui demande environ 25 à 30 Go d'espace disque. Avant de commencer, nous avons vérifié que notre machine répondait à ces exigences, puis nous l'avons préparée.

0.1 Vérification de la machine

Nous avons relevé les caractéristiques de la machine hôte à l'aide des commandes `lsb_release`, `free`, `nproc` et d'une recherche des indicateurs `vmx`/`svm` (virtualisation matérielle Intel/AMD) dans `/proc/cpuinfo`.

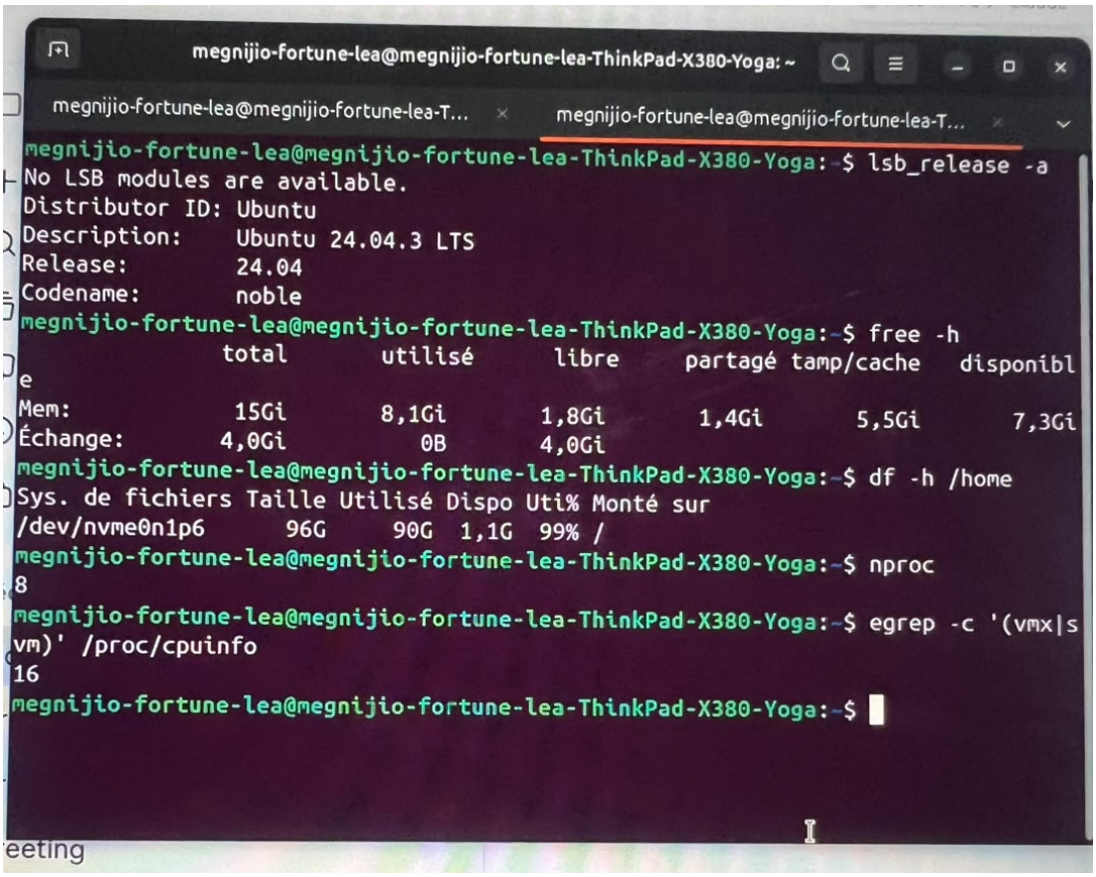


FIGURE 1 – Caractéristiques de la machine hôte

La machine hôte fonctionne sous **Ubuntu 24.04.3 LTS** et dispose de **15 Go de mémoire vive** et de **8 cœurs logiques**, ce qui est suffisant pour faire tourner une machine virtuelle et compiler le noyau. Le processeur prend également en charge la **virtualisation matérielle** (présence des indicateurs `vmx`/`svm`). En revanche, l'espace disque libre n'était que de **1,1 Go (99 % d'utilisation)**, ce qui était insuffisant. La machine convient donc, à l'exception de l'espace disque, qu'il a fallu libérer.

0.2 Analyse et libération d'espace

Les commandes `lsblk` et `du` ont montré que le disque contient une partition Ubuntu (ext4, 98 Go) presque pleine et une partition Windows (NTFS, 376 Go) disposant de 264 Go libres. Nous avons ensuite supprimé uniquement des données inutiles ou reconstructibles :

- deux anciennes machines virtuelles GNS3 inutilisées (environ 7 Go) ;
- seize anciennes révisions d'applications Snap et leurs copies dans le cache de Snap (environ 5 Go) ;
- les caches de VS Code, IntelliJ et pip (environ 3,5 Go) ;
- les anciens journaux système avec `journalctl --vacuum-size=200M` (652 Mo).

L'espace libre est ainsi passé à 17 Go, ce qui restait insuffisant.

0.3 Déplacement du Bureau vers la partition Windows

Nous avons déplacé le dossier Bureau (15,6 Go) vers la partition Windows :

1. désactivation du *démarrage rapide* de Windows, qui laisse la partition NTFS en état de veille et rend l'écriture depuis Linux risquée ;
2. vérification avec `findmnt` que la partition est montée en lecture-écriture (`rw`) ;
3. copie avec `rsync -rlt` (dossiers, liens et dates, sans les permissions Linux que NTFS ne gère pas) : 10 227 fichiers en 9 min 20 s ;
4. vérification de la copie (même nombre d'éléments, 12 016 de chaque côté, et empreintes identiques avec `rsync -c`), puis suppression des originaux.

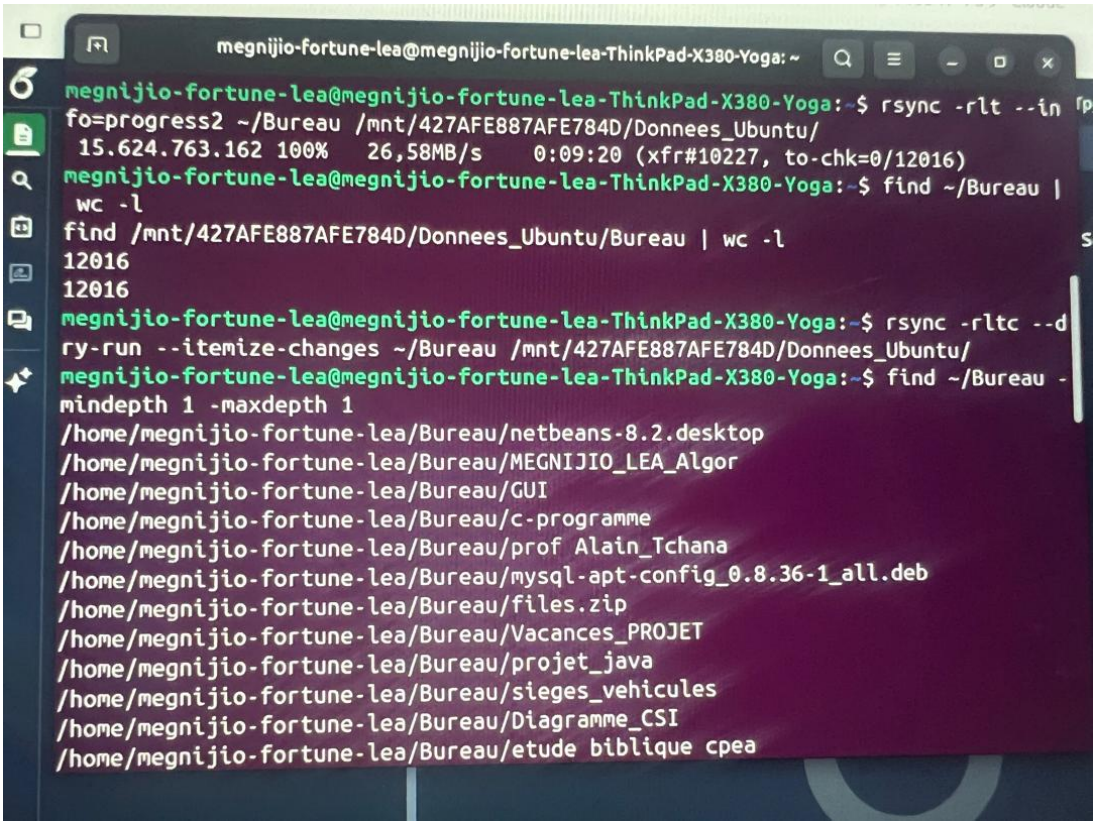


FIGURE 2 – Caractéristiques de la machine hôte

0.4 Difficultés rencontrées

— **Langue du système.** Le filtre `awk '/disabled/'` ne trouvait aucune révision Snap, car le système en français affiche `désactivé`. j'ai adapté la commande.

— **Espace non libéré.** Après la suppression des révisions Snap, l'espace libre n'avait pas changé. Le cache de Snap contenait des *liens physiques* vers les mêmes données ; or un fichier n'est réellement effacé que lorsque son dernier lien disparaît (compteur de liens de l'inode à zéro). Nous avons donc supprimé les fichiers du cache dont le compteur valait 1.

0.5 Bilan

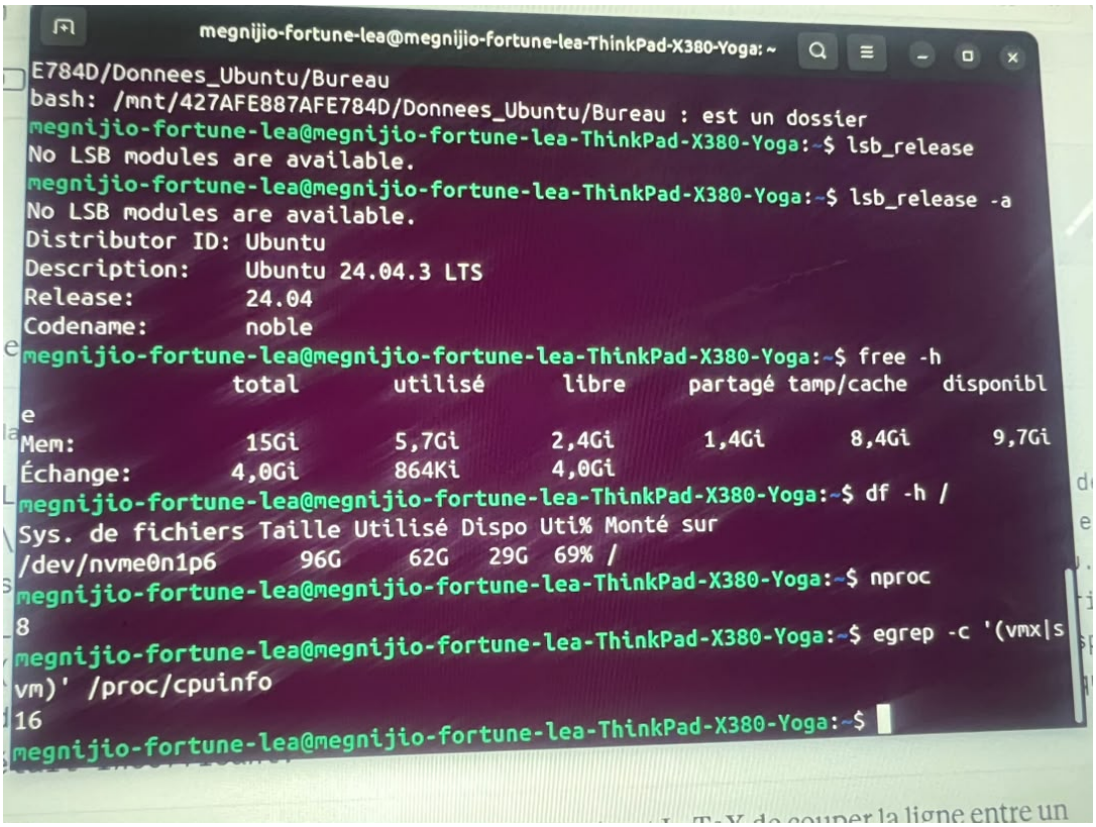


FIGURE 3 – Caractéristiques de la machine hôte

L'espace libre sur la partition Ubuntu est passé de **1,1 Go à 32 Go** (de 99 % à 66 % d'utilisation), sans perte de données personnelles. La machine est prête pour la création de la machine virtuelle.

Chapitre 1

TP1 – Compilation du noyau Linux et exploration du disque

1.1 Objectifs du TP

— Compiler un noyau Linux (version 5.4.1) à partir de son code source, l'installer et démarrer dessus.
— Extraire le binaire du noyau installé et y retrouver le code d'une fonction du code source.
— Observer l'organisation de la mémoire physique.
— Manipuler les disques : partitions, formatage, montage, sauvegarde et restauration.

1.2 Notions clés

— **Noyau** : cœur du système d'exploitation, seul programme à accéder directement au matériel. Il gère les processus, la mémoire, les périphériques, et répond aux appels système des programmes.
— **Machine virtuelle** : ordinateur simulé par un hyperviseur (ici VirtualBox), qui permet de remplacer le noyau sans risque pour la machine hôte.
— **Compilation** : transformation du code source (C) en binaire exécutable. Pour le noyau, `make` orchestre la compilation de milliers de fichiers ; l'option `-j` les compile en parallèle.
— **Module** : partie du noyau compilée à part (fichier `.ko`) et chargée seulement si nécessaire, par exemple un pilote.
— **GRUB et initramfs** : GRUB est le chargeur d'amorçage, qui charge le noyau au démarrage ; l'initramfs est un petit système de fichiers temporaire contenant les modules nécessaires pour accéder au disque.
— **`vmlinuz`, `vmlinux` et `System.map`** : le noyau compressé, le noyau décompressé (format ELF), et la table des adresses des fonctions du noyau.
— **`/proc/iomem`** : fichier virtuel généré par le noyau, qui décrit l'espace d'adresses physiques (RAM, noyau, périphériques).

1.3 Adaptations par rapport au sujet

— **Virtualisation** : le sujet utilise une VM Xen (pont `xenbr0`, adresse `10.0.0.2`). Nous avons utilisé une VM VirtualBox en mode NAT, avec une redirection du port 2222 de l'hôte vers le port 22 (SSH) de la VM.
— **Partage NFS** : non réalisé. Il servait à pallier un disque de VM de 5 Go ; le disque de notre VM fait 25 Go, ce qui suffit pour stocker et compiler le code source.
— **Système invité** : Ubuntu Server 20.04, dont le compilateur (gcc 9.4) est contemporain du noyau 5.4.1. Un compilateur récent provoque des erreurs sur ce code de 2019.
— **Configuration** : la configuration d'Ubuntu a été ajustée en retirant les certificats propres à Ubuntu (absents du code de kernel.org) et les informations de débogage (`DEBUG_INFO`), qui auraient porté la compilation à plus de 20 Go.

1.4 Partie 1 : compilation et exploration du noyau

1.4.1 Création de la machine virtuelle

La VM a été créée avec `VBoxManage` : 6 Go de RAM, 6 cœurs et un disque dynamique de 25 Go. Ubuntu Server 20.04.6 y a été installé à partir de l'ISO officielle, dont l'intégrité a été vérifiée par son empreinte SHA-256. Le serveur SSH permet de travailler dans la VM depuis le terminal de l'hôte. Avant toute modification, le noyau en service était le **5.4.0-216-generic**.

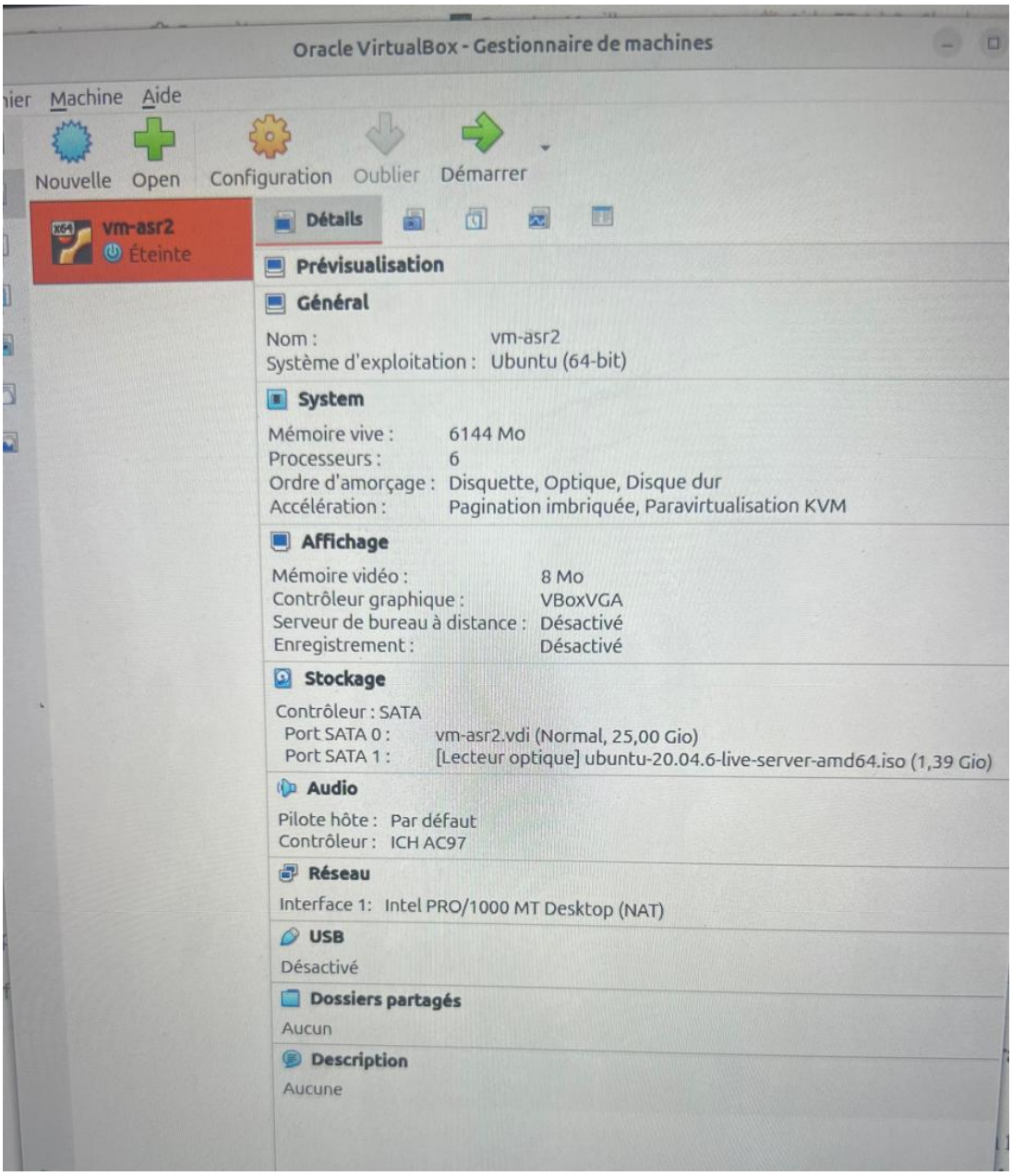


FIGURE 1.1 – Machine Virtuelle vm-asr2

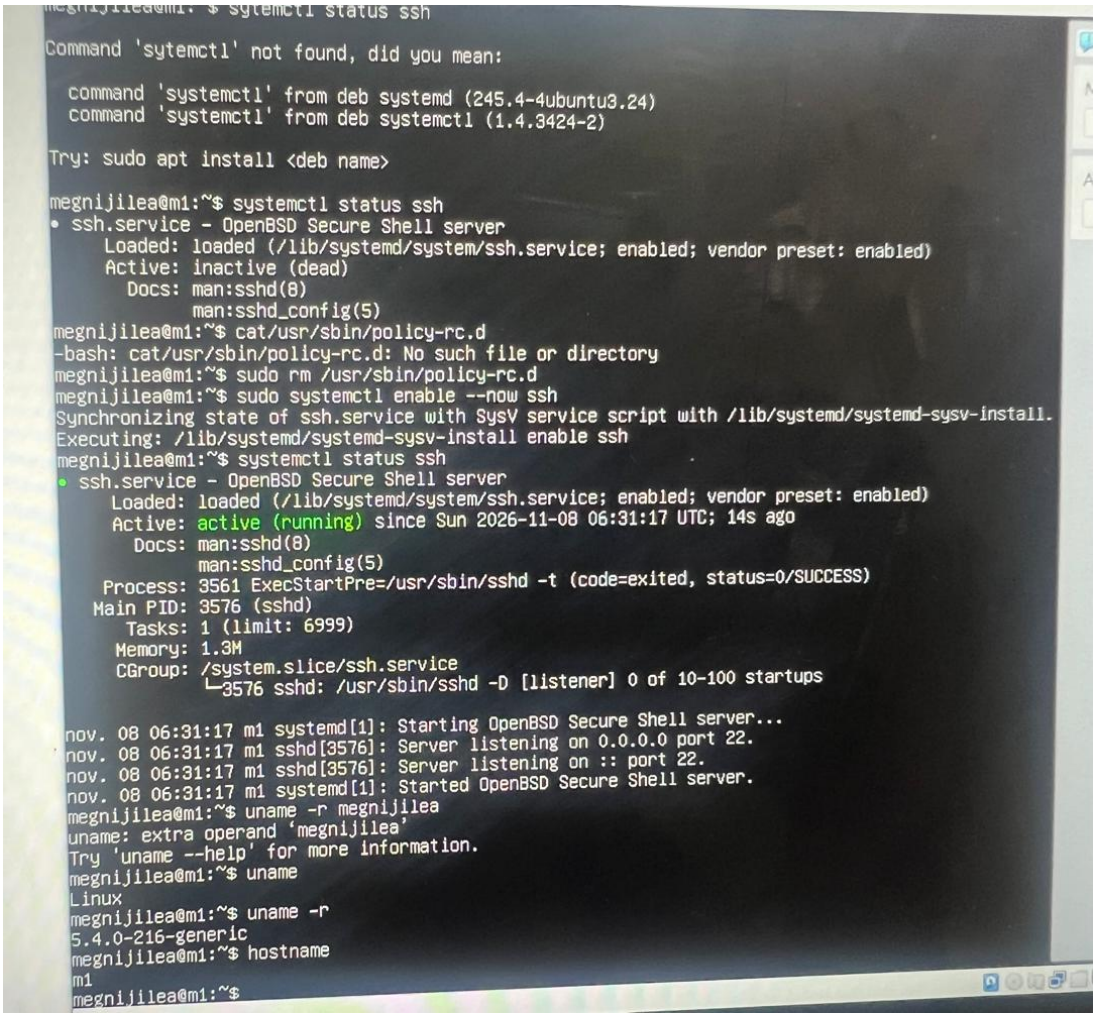


FIGURE 1.2 – Connexion SSH à la VM et noyau d'origine

1.4.2 Compilation

Le code source du noyau 5.4.1 a été téléchargé depuis kernel.org dans `/usr/src/linux5.4.1`, vérifié (SHA-256), puis décompressé. Nous avons repris la configuration du noyau en service (`/boot/config-5.4.0-216-generic`), puis nous l'avons adaptée (`scripts/config`, `make olddefconfig`). Après installation des outils de compilation (`build-essential`, `bc`, `flex`, `bison`, `libssl-dev`, `libelf-dev`), la compilation a été lancée avec `make -j6`.

Elle a duré **96 min 46 s** (temps réel), pour 522 min de calcul cumulé sur les cœurs : en moyenne, **5,4 cœurs** ont travaillé en parallèle. Sans parallélisme, il aurait fallu environ 9 heures.

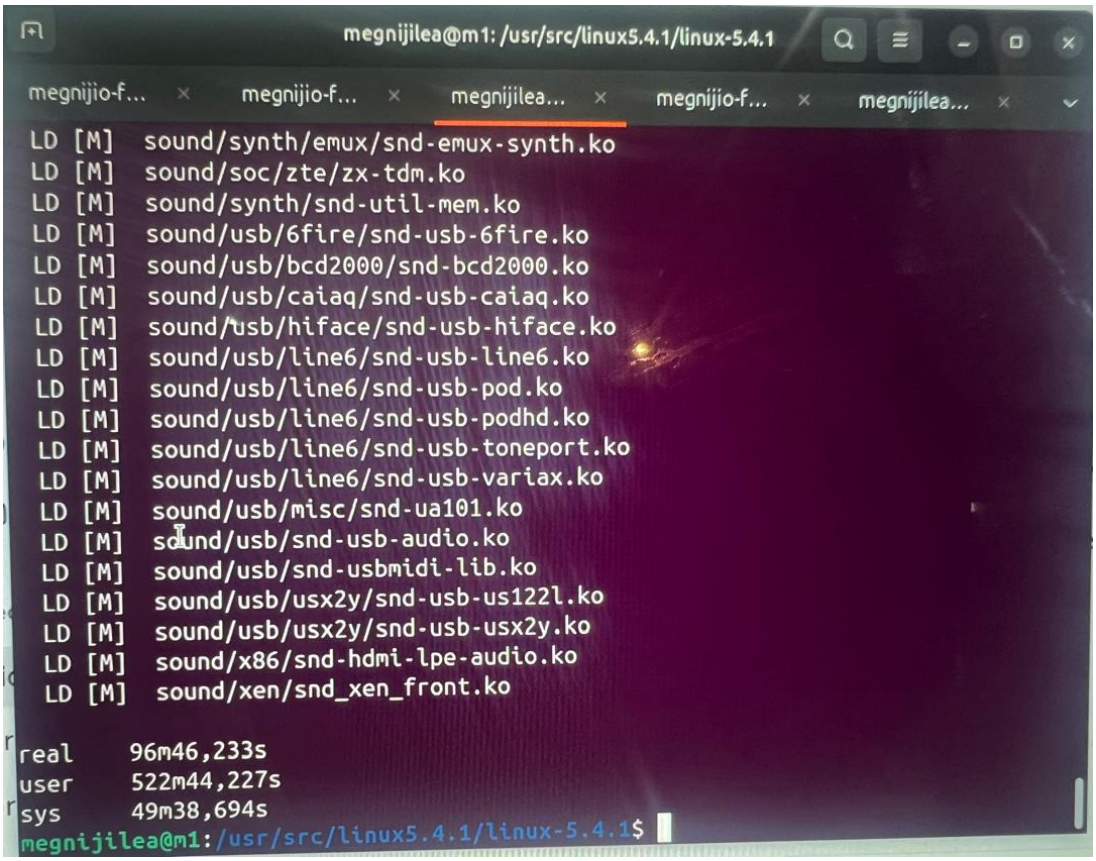


FIGURE 1.3 – Fin de la compilation et temps mesurés

1.4.3 Installation et vérification

Comme le demande le sujet, j'ai avons vérifié avant l'installation que ni `/boot` ni `/boot/grub/grub.cfg` ne contenaient de référence au noyau 5.4.1. Les modules ont ensuite été installés dans `/lib/modules/5.4.1` (`make modules_install`), puis le noyau (`make install`). Cette dernière commande a copié dans `/boot` le noyau (`vmlinuz-5.4.1`), sa table des symboles (`System.map-5.4.1`) et sa configuration, a généré l'initramfs, et a mis à jour GRUB.

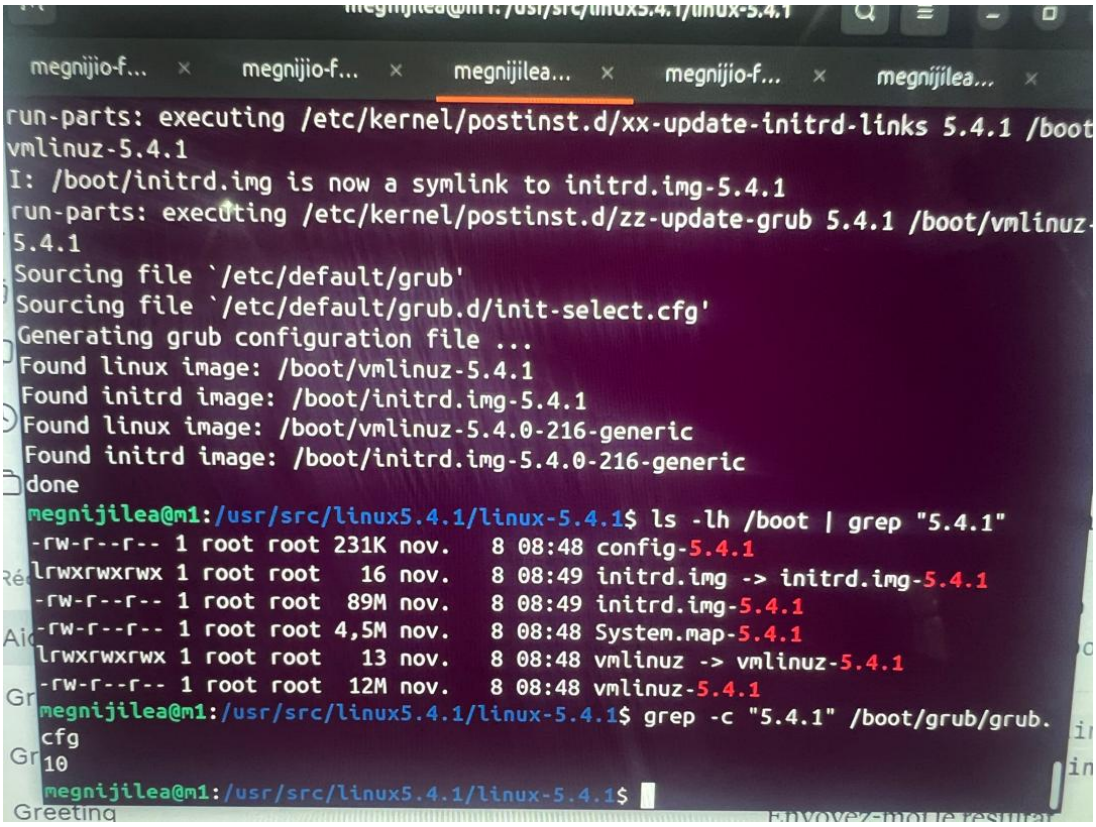


FIGURE 1.4 – Fichiers du noyau 5.4.1 dans /boot et entrées GRUB

Après redémarrage, `uname -a` confirme que la VM fonctionne sur **notre noyau**, compilé le jour même (`#1`).

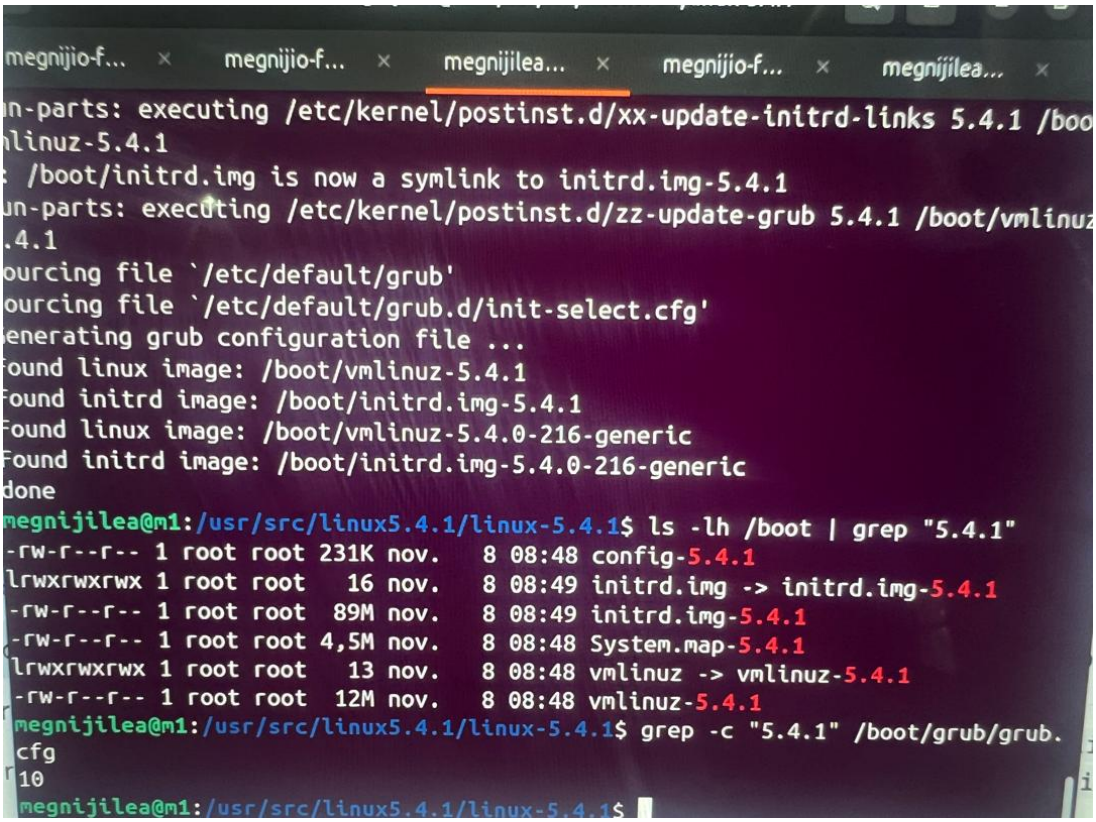


FIGURE 1.5 – La VM fonctionne sur le noyau 5.4.1 compilé

1.4.4 Extraction du binaire et recherche de `tcp_v4_do_rcv`

Le noyau installé (`vmlinuz-5.4.1`, 12 Mo) a été décompressé avec le script `extract-vmlinux` fourni avec le code source, ce qui donne un exécutable ELF de 43 Mo. Ce binaire est *stripped* : les noms des fonctions en ont été retirés. L'adresse de `tcp_v4_do_rcv` a donc été cherchée dans `System.map-5.4.1` (`ffffffff819c76e0`), puis son code assembleur a été affiché avec `objdump -d`.

Les adresses des fonctions appelées, retrouvées à leur tour dans `System.map`, correspondent à `__skb_checksum_complete` et à `kfree_skb`. On reconnaît ainsi dans l'assembleur la fin de la fonction C : si le paquet est corrompu, deux compteurs d'erreurs sont incrémentés (`incq`), le paquet est libéré (`kfree_skb`) et la fonction retourne 0 (`xor %eax,%eax`).

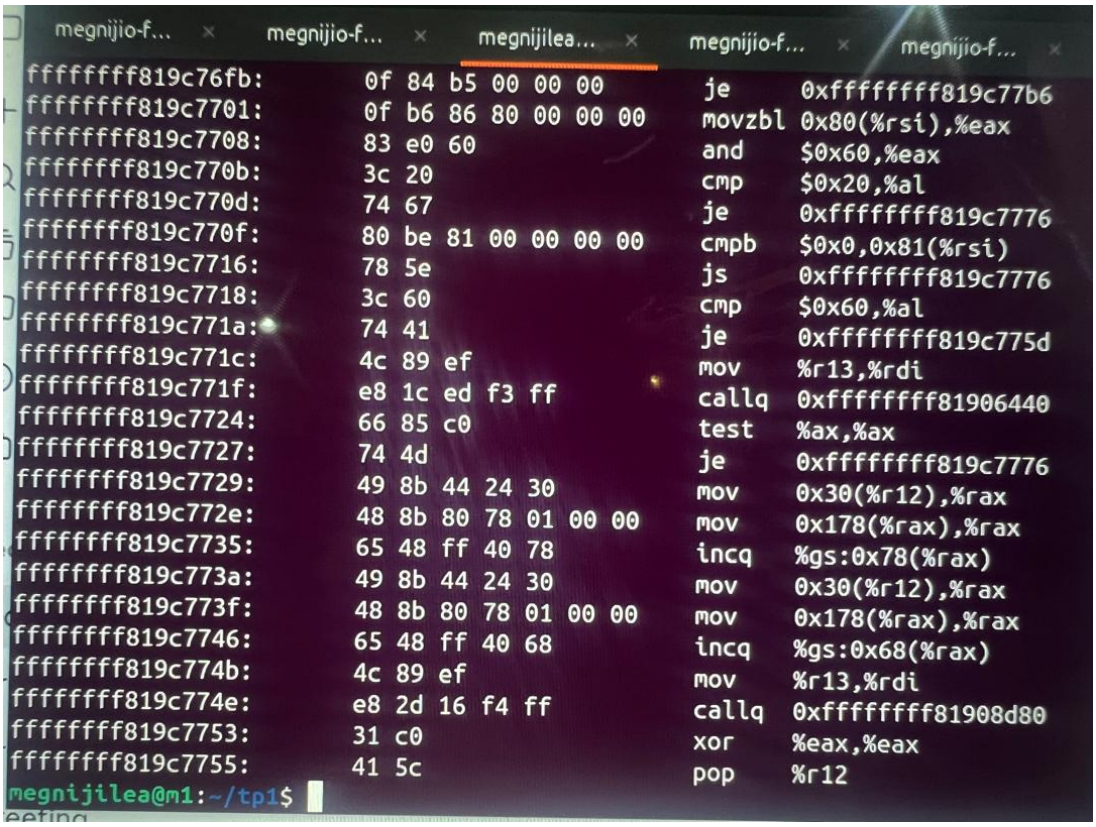


FIGURE 1.6 – Adresse de tcp_v4_do_rcv, désassemblage et fonctions appelées

1.4.5 Cartographie de la mémoire physique

Le fichier `/proc/iomem` (lu avec `sudo`, car le noyau masque les adresses aux utilisateurs) montre que la RAM totale, environ 6 Go, est découpée en trois zones : une zone historique sous 1 Mo, environ 3,5 Go sous la barre des 4 Go, et 2,5 Go au-delà. Ce découpage s'explique par le **trou PCI**, une plage d'adresses réservée aux périphériques. Le noyau occupe environ 29 Mo de RAM (code, données, bss). Son adresse physique varie à chaque démarrage à cause du **KASLR**, un placement aléatoire destiné à compliquer les attaques.

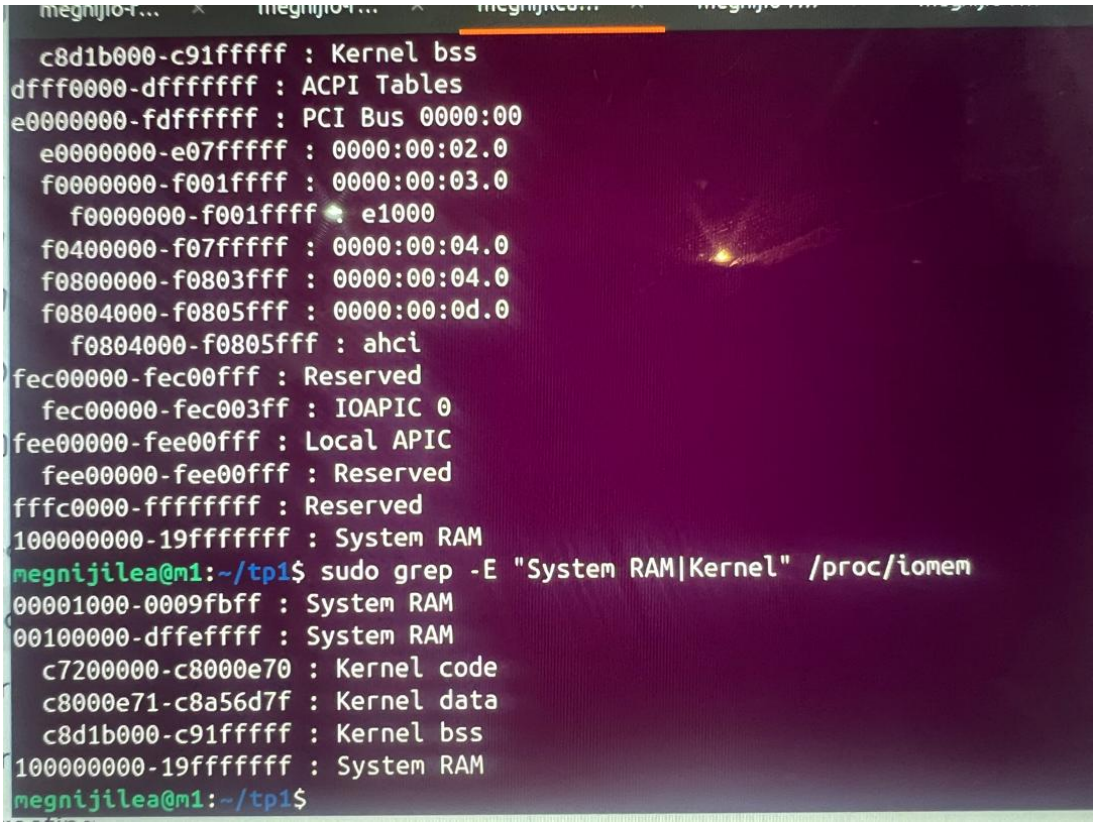


FIGURE 1.7 – La RAM et le noyau dans /proc/iomem

1.5 Partie 2 : exploration du disque

1.5.1 Ajout d'un disque de test

Comme dans le sujet, nous avons créé avec `dd` une image brute de 1 Go remplie de zéros (`if=/dev/zero bs=1M count=1024`). Nous l'avons convertie au format VirtualBox (`VBoxManage convertfromraw`), puis branchée sur la VM. Le disque apparaît dans la VM sous le nom **`/dev/sdb`** (et non `/dev/xvdb`, nom donné par Xen), sans aucune table de partitions. La commande `fdisk -l` montre aussi la structure du disque principal `/dev/sda` : une table GPT, une partition *BIOS boot* de 1 Mo pour GRUB, et une partition de 25 Go pour le système.

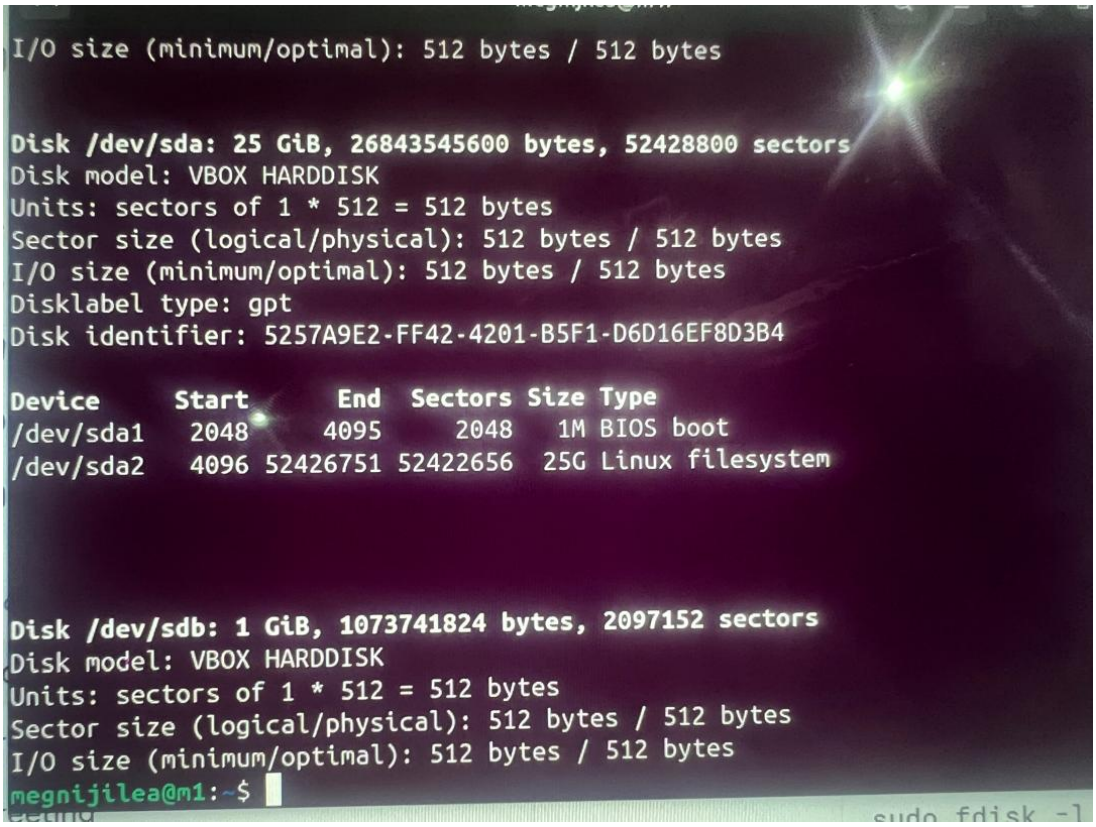


FIGURE 1.8 – Les deux disques de la VM vus par fdisk

1.5.2 Partitionnement et formatage

Avec `fdisk /dev/sdb`, nous avons créé une table de partitions MBR (`dos`) et une partition principale `/dev/sdb1`, qui occupe tout le disque (du secteur 2048 à la fin, soit 1023 Mo). Pour enregistrer la table, il faut terminer par `w` : la commande `q` indiquée dans le sujet quitte **sans enregistrer**. La partition a ensuite été formatée en ext3 (`mkfs.ext3`) : 261 888 blocs de 4 Ko, 65 536 inodes, des copies de secours du superbloc et un journal de 16 Mo.

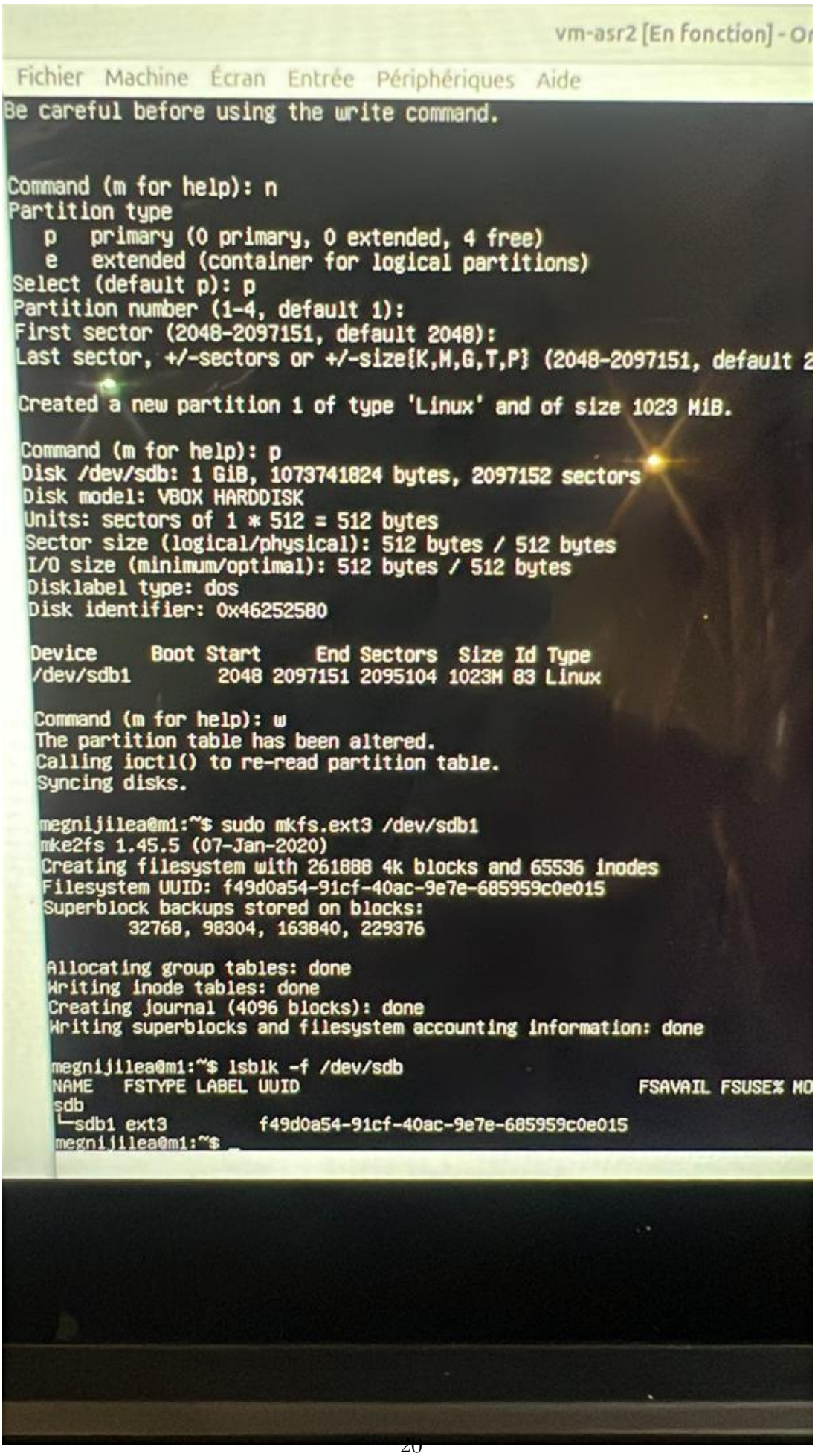


FIGURE 1.9 – Création de la partition et formatage en ext3

1.5.3 Montage et fichier `/etc/fstab`

La partition a été montée sur `/mnt/new_partition` (`mount`), puis utilisée pour écrire et relire un fichier. Nous avons vérifié la note du sujet : on monte une **partition**, pas un disque, car le début du disque contient la table des partitions et non un système de fichiers.

Pour que le montage soit automatique, nous avons ajouté la ligne suivante dans `/etc/fstab` :

```
UUID=f49d0a54-91cf-40ac-9e7e-685959c0e015 /mnt/new_partition ext3 nofail 0 2
```

L'UUID désigne la partition de façon stable, et l'option `nofail` évite qu'une partition absente bloque le démarrage. La ligne a été testée avec `mount -a` avant le redémarrage. Après redémarrage, la partition est bien montée automatiquement.

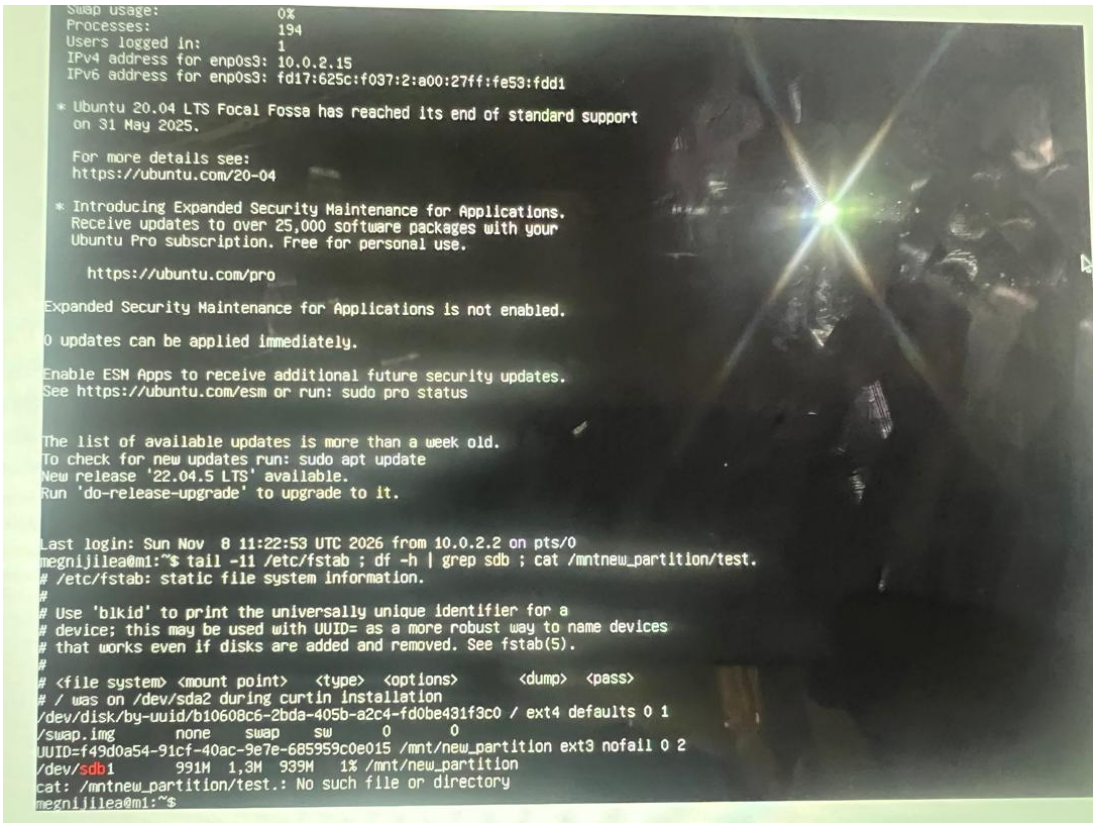


FIGURE 1.10 – Montage automatique de la partition après redémarrage

1.5.4 Sauvegarde, destruction et restauration du MBR avec `dd`

Le disque de VirtualBox (`.vdi`) commence par un en-tête propre à VirtualBox : écrire des zéros au début du fichier n'aurait pas effacé le MBR. Nous avons donc branché le disque de la VM sur l'hôte avec **`qemu-nbd`**, qui le présente comme un disque brut (`/dev/nbd0`). Par précaution, une copie complète du `.vdi` a été faite au préalable.

Le premier secteur (512 octets) contient le **MBR** : le code d'amorçage de GRUB (on y lit ses messages d'erreur), une entrée de partition de type `ee` (GPT de protection) et la signature `55 AA`. Nous l'avons sauvegardé (`dd ... bs=512 count=1`), puis remplacé par des zéros.

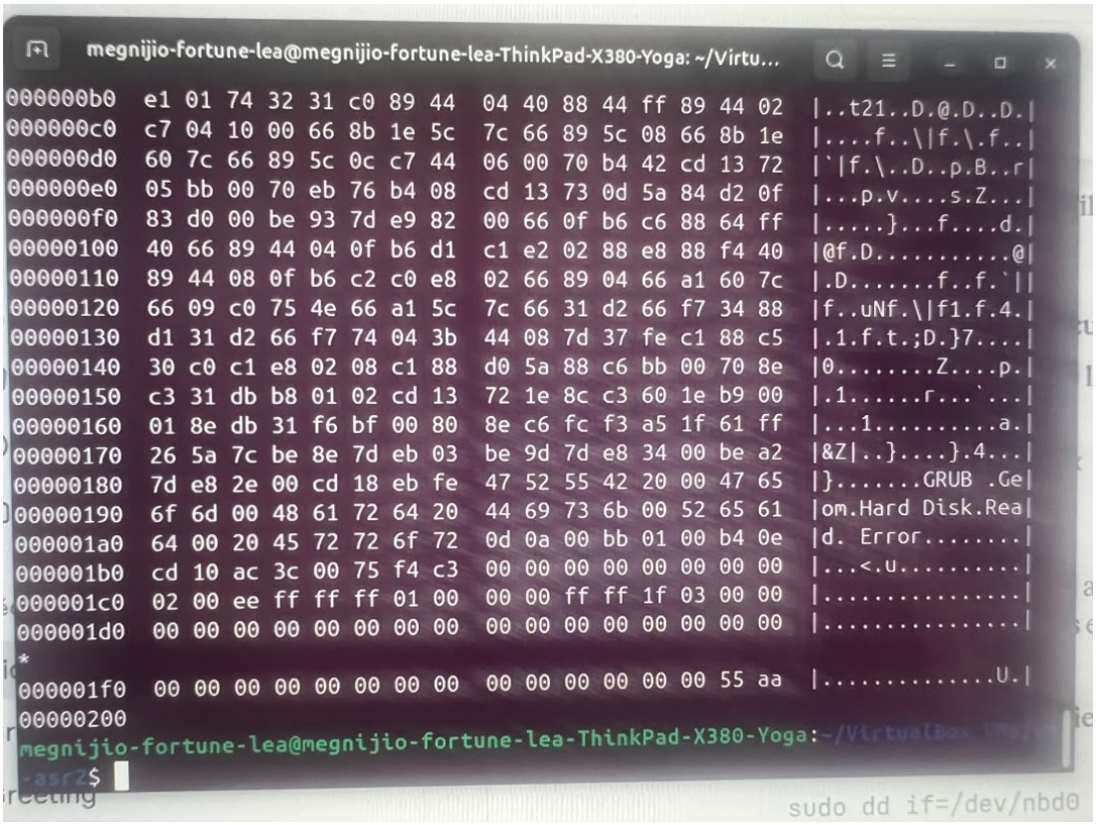


FIGURE 1.11 – Le MBR du disque de la VM en hexadécimal

La VM ne démarre alors plus : le BIOS ne trouve plus de disque démarrable. Il a suffi d'effacer 512 octets, alors que le reste du disque est intact.

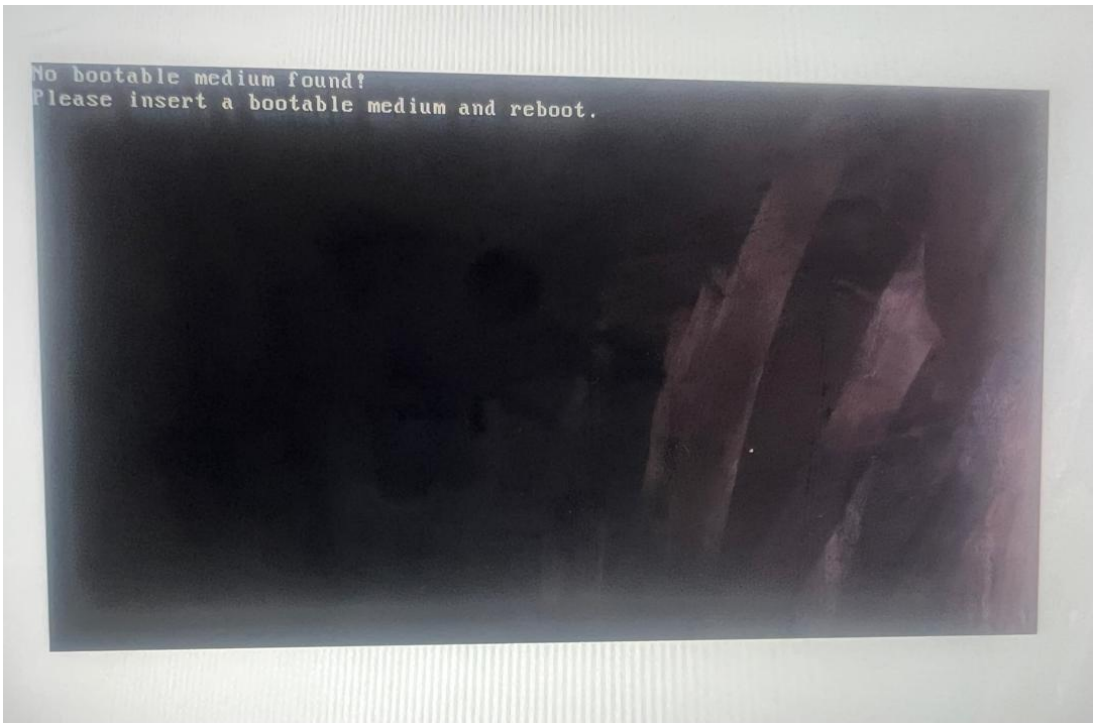


FIGURE 1.12 – Échec du démarrage après l'effacement du MBR

Après restauration du secteur sauvegardé (`dd` en sens inverse), la VM redémarre normalement, sur le noyau 5.4.1 et avec la partition de test montée.

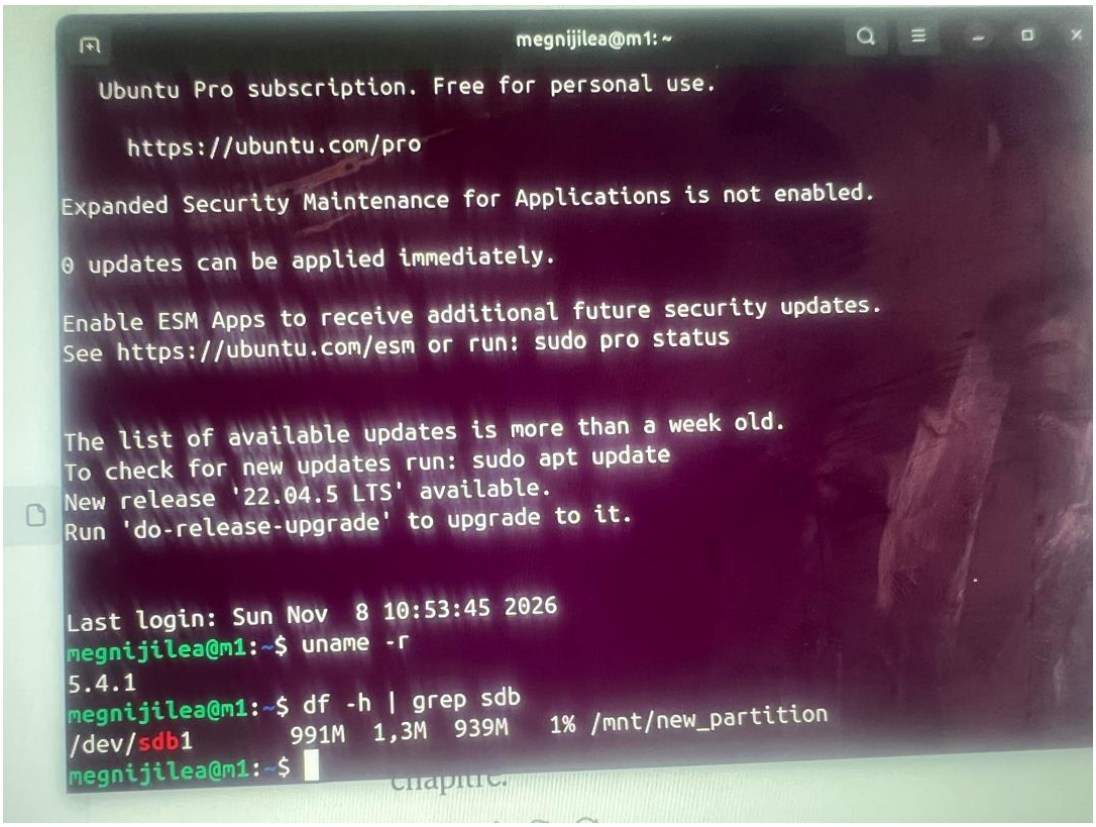


FIGURE 1.13 – La VM fonctionne à nouveau après la restauration du MBR

1.6 Difficultés rencontrées et solutions

— **Installation interrompue.** L'annulation des mises à jour en fin d'installation a laissé une source CD-ROM dans `/etc/apt/sources.list` et un fichier `/usr/sbin/policy-rc.d` qui empêchait tout service de démarrer, dont le serveur SSH. Nous avons désactivé la source et supprimé le fichier.
— **Compilation du noyau officiel avec la configuration d'Ubuntu.** Il a fallu retirer les certificats propres à Ubuntu et désactiver `DEBUG_INFO`.
— **Syntaxe de `/etc/fstab`.** Plusieurs erreurs (espace après une virgule, ligne coupée en deux, UUID incomplet) nous ont appris que les champs sont séparés par des espaces, que `nofail` rend une erreur d'UUID silencieuse, et qu'il faut vérifier avec `mount -a` et `findmnt --verify`, après avoir sauvegardé le fichier.
— **Démontage refusé.** Ubuntu montait automatiquement la partition de la VM branchée par `qemu-nbd`. L'outil `fuser` a montré que le gestionnaire de fichiers (`nautilus`) l'utilisait encore.

1.7 Conclusion du TP

Ce TP nous a permis de parcourir le chemin complet d'un noyau Linux : du code source au binaire (compilation, modules, installation, démarrage via GRUB et l'initramfs), puis du binaire au code source (extraction et désassemblage de `tcp_v4_do_rcv`). Nous avons aussi manipulé les couches de stockage, de la table des partitions au système de fichiers et à son montage. L'expérience sur le MBR montre enfin le rôle critique des tout premiers octets d'un disque dans le démarrage d'une machine.

Chapitre 2

TP2 – Programmation d'un mini-shell

2.1 Objectifs du TP

Réaliser en C un mini-shell, c'est-à-dire un programme qui affiche une invite, lit des commandes et les exécute. Il doit gérer :

— des commandes internes : `exit`, `ps`, `export`, `env`, `kill`, `bg` ;
— l'exécution de programmes externes, à l'aide de sa propre variable `PATH` ;
— l'exécution en arrière-plan (`&`) et la commande `nohup` ;
— l'interruption (Ctrl-C) et la suspension (Ctrl-Z) du processus au premier plan.

Ce TP permet de maîtriser les fonctions C de création et de gestion des processus.

2.2 Notions clés

— **Processus** : programme en cours d'exécution, identifié par un numéro unique, le **PID**. Il connaît aussi le numéro de son parent, le **PPID**.
— `fork()` : duplique le processus appelant. Le fils est une copie du père (même code, même mémoire). `fork` renvoie **0 dans le fils** et le **PID du fils dans le père**, ce qui permet de leur faire exécuter des actions différentes.
— `execve()` : remplace le programme du processus par un autre programme. En cas de succès, il **ne revient jamais** ; le code qui le suit ne s'exécute qu'en cas d'échec.
— `wait()` / `waitpid()` : le père attend un changement d'état de son fils (fin, arrêt par un signal, suspension), puis récupère son statut. Les macros `WIFEXITED`, `WIFSIGNALED` et `WIFSTOPPED` permettent de l'interpréter.
— **Principe d'un shell** : pour chaque commande, il fait un `fork`, le fils exécute la commande avec `exec`, et le père attend sa fin avec `wait` avant de réafficher l'invite.

2.3 Déroulement – Partie 1

2.3.1 Étude du programme d'exemple `processus.c`

Avant d'écrire le shell, nous avons compilé (`gcc -Wall`) et exécuté le programme fourni `processus.c`. Il crée un fils avec `fork`. Le fils affiche une ligne de tirets, puis se transforme

en `ls -l /home` avec `execve`. Le père attend la fin du fils avec `waitpid` et affiche la façon dont il s'est terminé.

L'exécution montre que :

— le père (PID 40500) et le fils (PID 40501) s'exécutent en parallèle ;
— la **seconde ligne de tirets**, placée après l'`execve`, **ne s'affiche pas** : le fils a été remplacé par `ls`, et le code qui suivait a disparu avec l'ancien programme ;
— pour la même raison, le message final `Hello` n'est affiché qu'une fois, par le père ;
— le fils s'est terminé normalement, avec le code de retour 0 (succès de `ls`).

Un point, laissé dans le tampon de `printf` avant le `fork`, apparaît au début des premières lignes du père **et** du fils : `fork` a dupliqué la mémoire du processus, y compris ce tampon d'affichage non encore vidé.

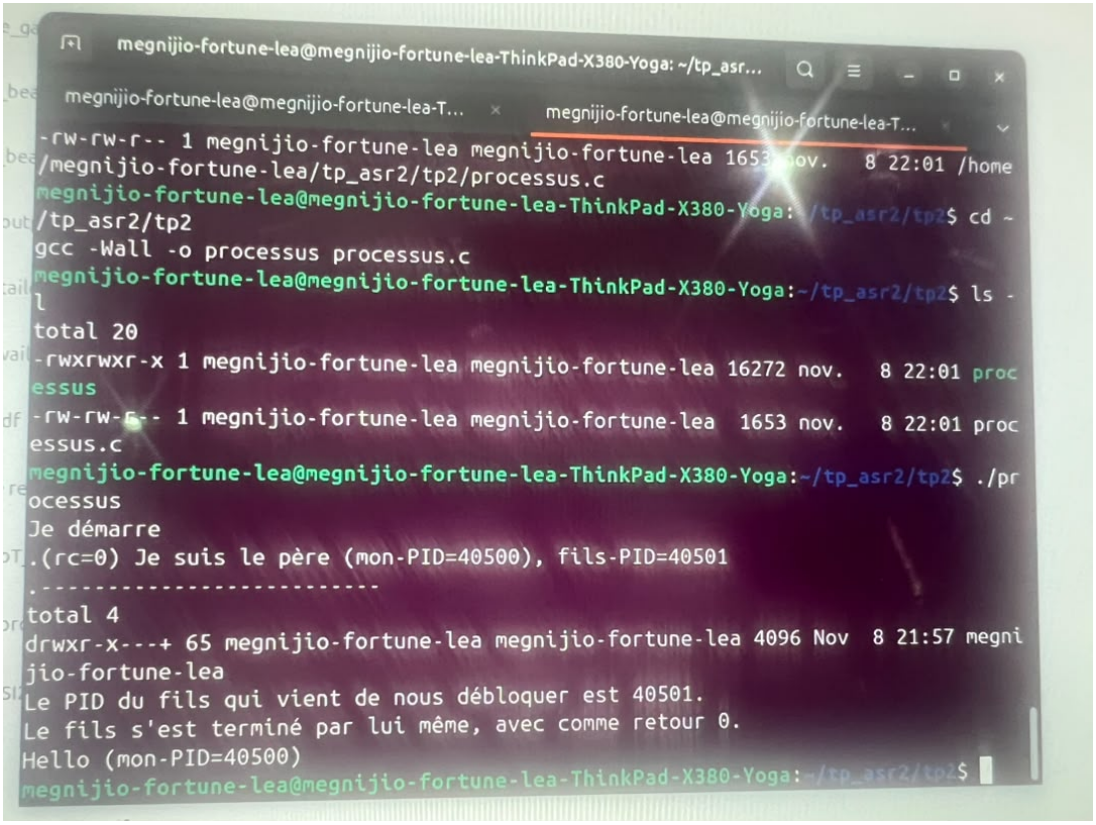


FIGURE 2.1 – Exécution du programme d'exemple processus.c

2.3.2 Boucle principale et lecture des commandes

Le shell tourne dans une boucle infinie : il affiche l'invite `mon-shell>`, lit une ligne avec `fgets`, puis la découpe en mots avec `strsep`. Les mots sont rangés dans un tableau `args` terminé par `NULL`, le format attendu par les fonctions `exec`. Les espaces multiples et les lignes vides sont ignorés. La commande interne `exit`, ou Ctrl-D (fin de saisie), quitte le shell. Comme le demande le sujet, toutes les déclarations (constantes, structures, fonctions) sont regroupées dans un fichier d'en-tête `minishell.h`, et chaque commande est implémentée dans sa propre fonction.

2.3.3 Exécution des commandes externes et table des processus

Pour une commande externe, le shell applique le schéma étudié avec `processus.c` : il crée un fils avec `fork`, le fils se transforme en la commande avec `execv`, et le père attend sa fin avec `waitpid` (exécution au premier plan). Si `execv` échoue, le fils affiche l'erreur avec `perror` et se termine avec le code 127.

Chaque processus créé est enregistré dans une **table des processus** : un tableau de structures `Processus` (PID, PPID, commande, état). Le père l'y ajoute après le `fork`, puis l'en retire quand le fils se termine. Cette table servira aux commandes `ps`, `kill` et `bg` de la partie 2. Le test `/bin/sleep 5` montre que le shell reste bloqué pendant que son fils s'exécute.

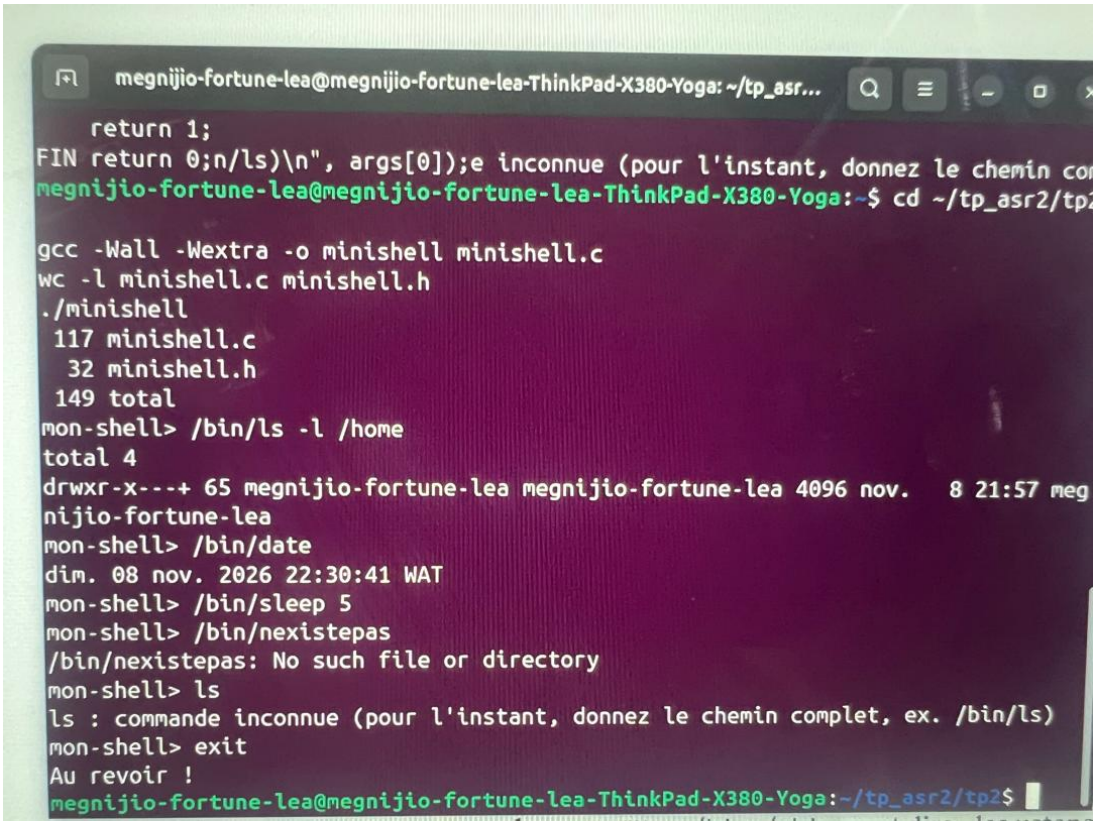


FIGURE 2.2 – Commandes externes lancées avec leur chemin complet

2.3.4 Variables d'environnement : `export` et `env`

Le shell gère ses **propres** variables d'environnement, indépendantes de celles du terminal qui l'a lancé. Elles sont stockées dans un tableau de structures `VarEnv` (nom, valeur). La commande `export NOM=VALEUR` repère le signe `=` avec `strchr`, sépare le nom de la valeur, puis ajoute la variable ou remplace sa valeur si elle existe déjà. La commande `env` affiche toutes les variables.

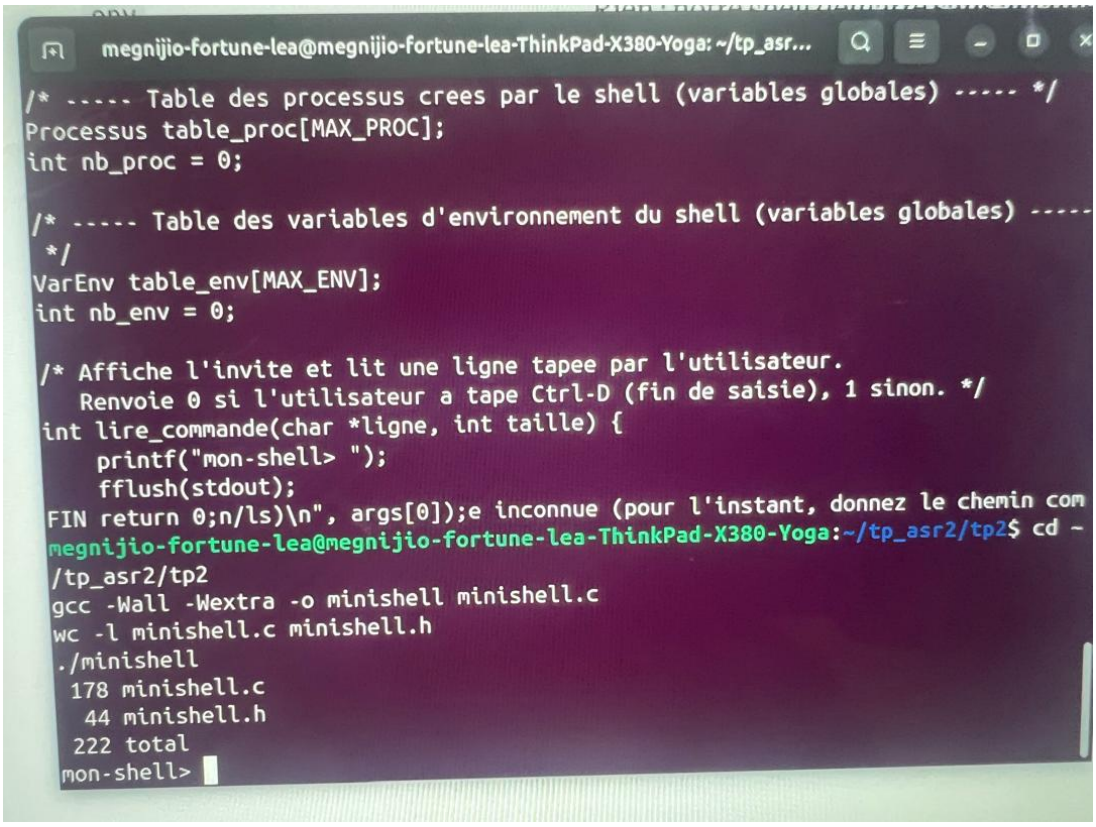


FIGURE 2.3 – Création, modification et affichage des variables du shell

2.3.5 Recherche des commandes dans le `PATH` du shell

Pour une commande donnée sans chemin (par exemple `ls`), le shell lit **sa** variable `PATH`, la découpe à chaque `:`, et construit pour chaque dossier un chemin candidat (`/bin/ls`, `/usr/bin/ls`...). La fonction `access(chemin, X_OK)` vérifie si ce fichier existe et est exécutable ; le premier trouvé est exécuté. Si `PATH` n'est pas défini, ou si aucun dossier ne contient la commande, un message d'erreur est affiché. Une commande contenant un `/` (`/bin/ls`, `./programme`) est exécutée directement, sans recherche.

Les tests montrent que `ls` est introuvable tant que `PATH` ne contient pas `/bin`, puis est trouvé dès qu'on l'y ajoute. Lancé depuis notre shell, le programme `processus` forme trois générations de processus : le shell, `processus`, puis son fils transformé en `ls`.

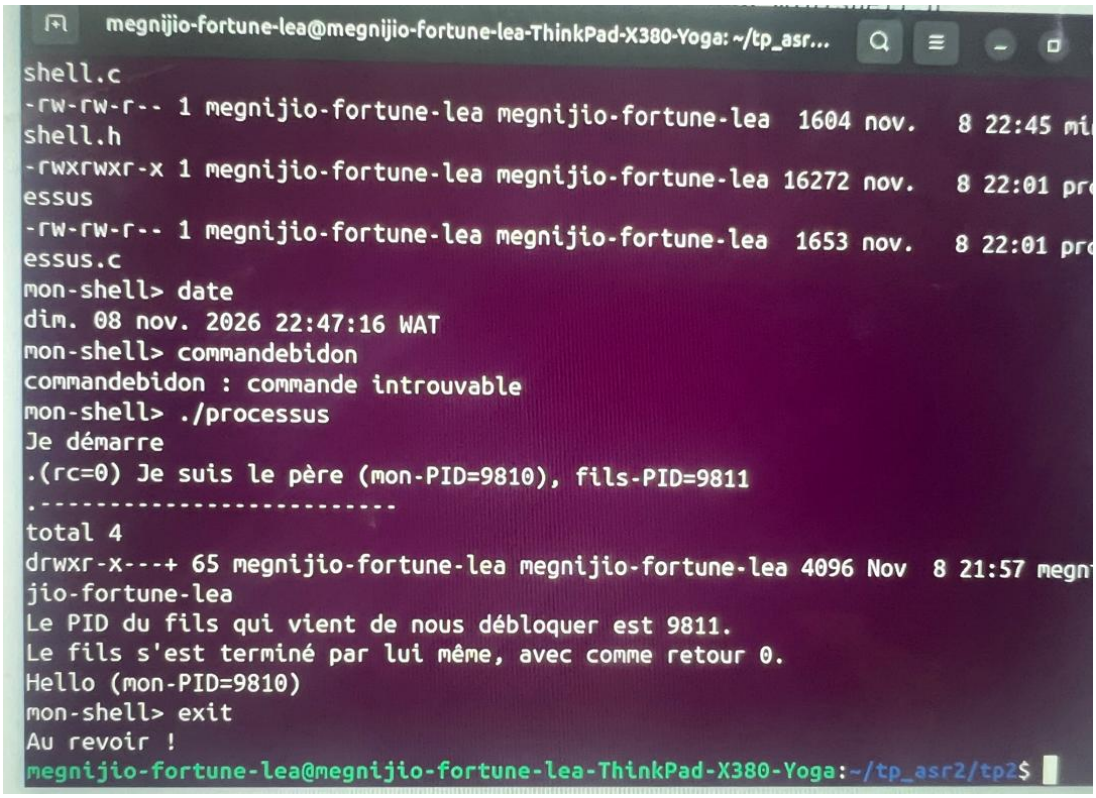


FIGURE 2.4 – Recherche des commandes dans le PATH du shell

2.4 Déroulement – Partie 2 : signaux et gestion des processus

2.4.1 Ctrl-C et Ctrl-Z

Le shell installe, avec `sigaction`, des gestionnaires pour `SIGINT` (Ctrl-C) et `SIGTSTP` (Ctrl-Z) : il ne s'arrête plus lui-même, et le signal atteint seulement le processus au premier plan. Le père attend ce processus avec `waitpid(..., WUNTRACED)`, qui se débloque aussi en cas de suspension : le processus reste alors dans la table, avec l'état « suspendu ».

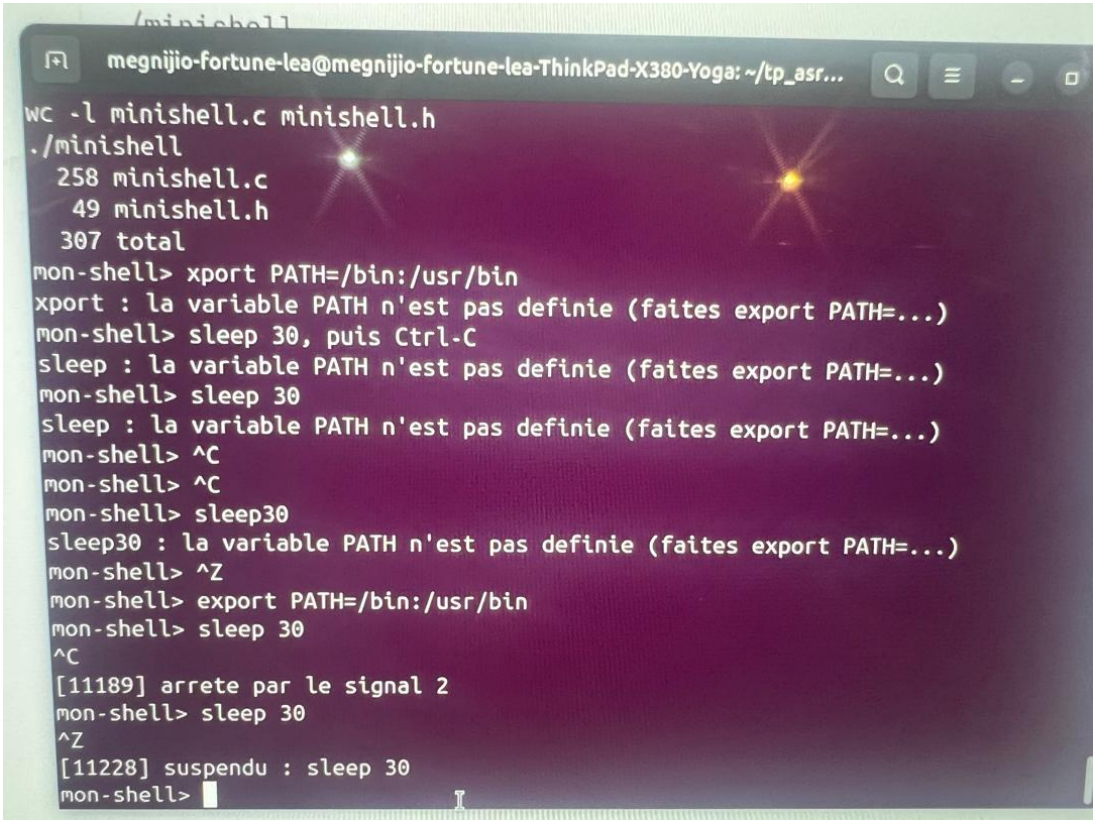


FIGURE 2.5 – Ctrl-C et Ctrl-Z sur un processus au premier plan

2.4.2 Arrière-plan et commandes internes

Une commande terminée par `&` est lancée sans attente. Le shell est prévenu de ses changements d'état par le signal `SIGCHLD`, dont le gestionnaire met à jour la table avec `waitpid(-1, ..., WNOHANG)`. Les commandes internes exploitent cette table :

- `ps` affiche les processus du shell (PID, PPID, état, commande) ;
- `kill -t | -s | -c PID` termine, suspend ou relance un processus (`SIGTERM`, `SIGSTOP`, `SIGCONT`) ; un processus relancé revient au premier plan ;
- `bg` relance en arrière-plan le dernier processus suspendu ;
- `nohup` lance une commande détachée du shell (elle ignore `SIGHUP` et n'est pas enregistrée) ;
- `exit` tue tous les processus rattachés au shell avant de quitter ; les commandes lancées avec `nohup` survivent.

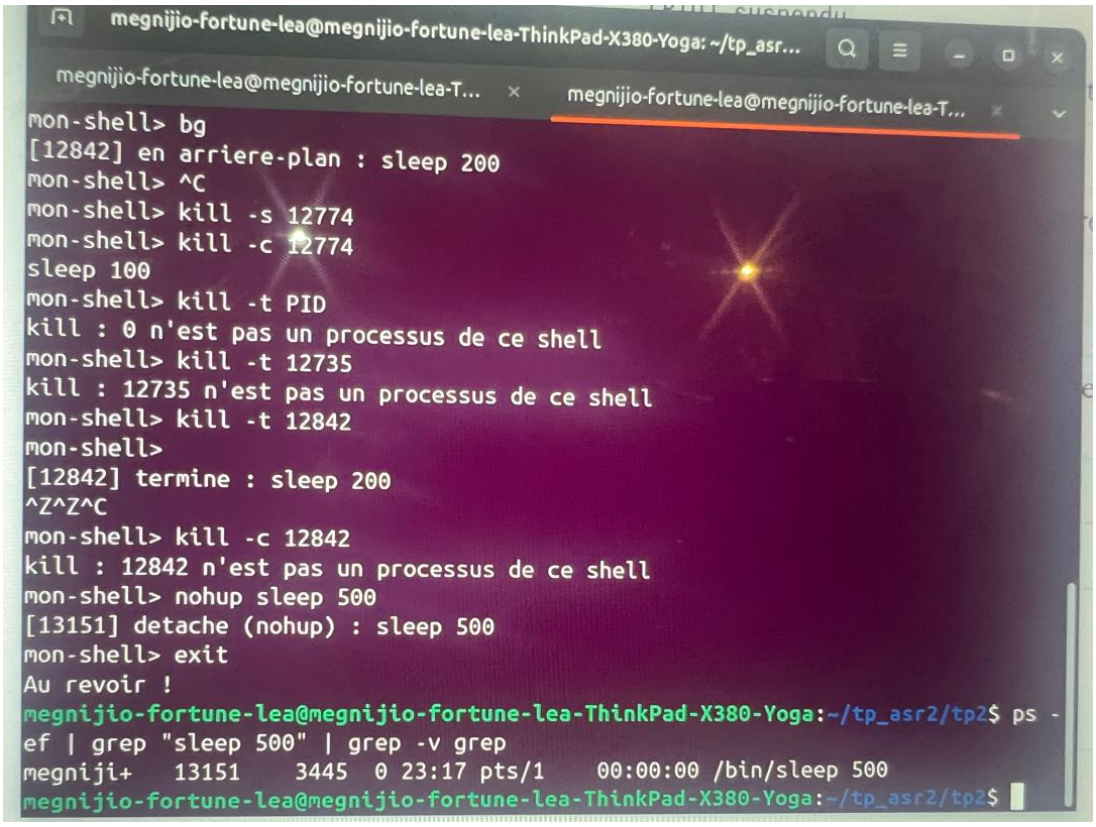


FIGURE 2.6 – Arrière-plan, ps, bg, kill, et survie du processus nohup après exit

2.5 Difficultés rencontrées et solutions

— **Ctrl-C tuait les processus en arrière-plan.** Tous les fils partageaient le groupe de processus du shell, et le terminal envoie Ctrl-C à tout le groupe. Chaque fils reçoit donc son propre groupe (`setpgid`), et le shell donne le terminal au processus du premier plan (`tcsetpgrp`), puis le reprend.

— **Concurrence entre `SIGCHLD` et l'attente au premier plan.** Le gestionnaire de `SIGCHLD` pouvait récupérer le fils attendu au premier plan. `SIGCHLD` est donc bloqué (`sigprocmask`) pendant cette attente.

— **Un processus suspendu ignore `SIGTERM`.** Pour le terminer, `kill -t` lui envoie `SIGTERM` puis `SIGCONT`.

2.6 Conclusion du TP

Ce TP nous a fait reconstruire le fonctionnement d'un shell : création de processus (`fork`, `exec`, `wait`), environnement propre (`PATH`), et contrôle des tâches par les signaux. Il montre aussi le rôle des groupes de processus et du terminal dans la gestion du premier et de l'arrière-plan. Les fichiers sources (`minishell.h` et `minishell.c`) sont remis séparément.

Chapitre 3

TP3 – Threads POSIX et ordonnancement

3.1 Objectifs du TP

Créer deux threads qui affichent leur nom toutes les 3 secondes, et un troisième thread jouant le rôle d'ordonnanceur : après 10 secondes, il impose toutes les 5 secondes une exécution alternée des deux threads. La partie notée demande de refaire cette alternance avec les contextes (`getcontext`, `makecontext`, `swapcontext`).

3.2 Notions clés

— **Thread** : fil d'exécution à l'intérieur d'un processus. Les threads d'un même processus partagent sa mémoire. Les threads `pthread` sont connus du noyau, qui les ordonnance en parallèle comme des processus.

— **Signal destiné à un thread** : `pthread_kill` envoie un signal à un thread précis. `SIGSTOP` ne convient pas, car il suspendrait tout le processus ; on utilise donc `SIGUSR1` pour suspendre (avec `sigsuspend`) et `SIGUSR2` pour reprendre.

— **Contexte** (`ucontext_t`) : état d'exécution sauvegardé (registres, pile, position dans le code). `swapcontext(a, b)` sauvegarde l'état courant dans `a` et reprend `b`. Les contextes permettent de programmer des « threads utilisateur », ordonnancés par le programme lui-même et non par le noyau.

3.3 Déroulement

3.3.1 Partie 1 : threads POSIX et signaux

Le programme crée deux threads de travail et un thread ordonnanceur avec `pthread_create`. Le gestionnaire de `SIGUSR1` appelle `sigsuspend` avec un masque qui n'accepte que `SIGUSR2` : le thread qui reçoit `SIGUSR1` reste bloqué jusqu'à ce que `SIGUSR2` le réveille. L'ordonnanceur attend 10 secondes, bloque le thread 2, puis, toutes les 5 secondes, envoie `SIGUSR1` au thread actif et `SIGUSR2` au thread bloqué.

L'exécution montre les deux phases attendues : pendant les 10 premières secondes, les deux threads s'affichent en parallèle (ordonnancés par le noyau) ; ensuite, ils s'exécutent en alternance par tranches de 5 secondes.

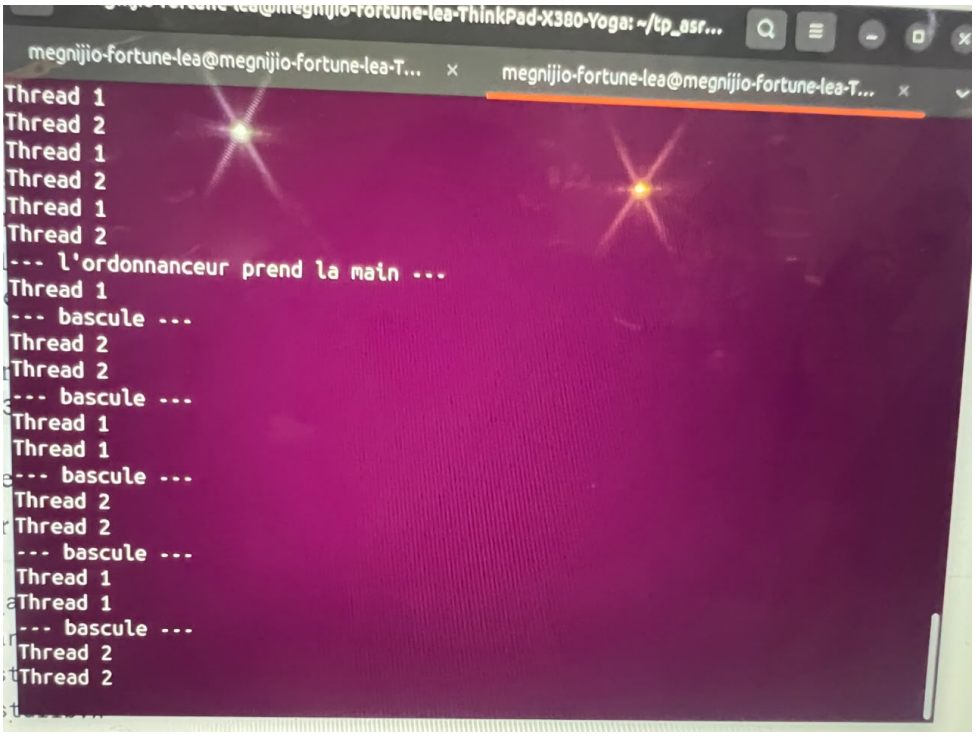


FIGURE 3.1 – Threads POSIX : exécution parallèle, puis alternée par l'ordonnanceur

3.3.2 Partie notée : les contextes

Chaque tâche reçoit son propre contexte, préparé avec `getcontext` puis `makecontext`, et sa propre pile de 64 Ko allouée avec `malloc`. L'ordonnanceur est le gestionnaire du signal `SIGALRM`, déclenché toutes les 5 secondes par `alarm` : il choisit l'autre tâche et y passe avec `swapcontext`, ce qui sauvegarde l'état de la tâche interrompue pour pouvoir la reprendre plus tard.

Il n'y a qu'un seul fil d'exécution : les tâches ne s'exécutent jamais en même temps, et l'alternance est visible dès le début. Comme le signal interrompt `sleep`, une tâche reprise peut afficher son nom avant la fin des 3 secondes, sans gêner l'alternance.

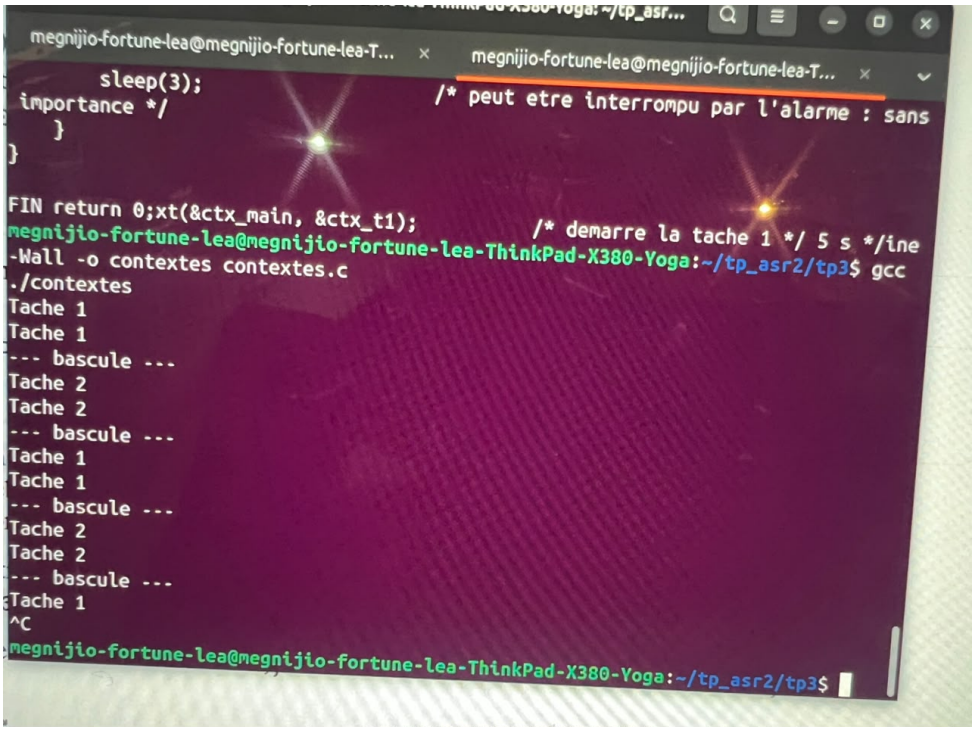


FIGURE 3.2 – Contextes : alternance des deux tâches toutes les 5 secondes

3.4 Conclusion du TP

Les threads POSIX sont ordonnancés par le noyau ; notre ordonnanceur ne peut que les bloquer et les débloquer par des signaux. Avec les contextes, le programme choisit lui-même la tâche à exécuter : c'est le principe d'un ordonnanceur de threads en espace utilisateur.

Chapitre 4

TP4 – Implémentation d'un appel système

4.1 Objectifs

Ajouter au noyau Linux 5.4.1 un appel système `make_vcpu(pid)` qui marque comme « vCPU » tous les threads d'un processus, en ajoutant un champ à la structure `task_struct`, puis l'appeler depuis un programme.

4.2 Notions clés

- **Appel système** : unique point d'entrée d'un programme vers le noyau. Le programme place un numéro et des arguments dans les registres du processeur, puis exécute l'instruction `syscall`, qui fait passer le processeur en mode privilégié.
- **Table des appels système** (`syscall_64.tbl`) : associe chaque numéro à une fonction du noyau.
- `task_struct` : structure décrivant chaque tâche. Pour le noyau, un thread est une tâche à part entière, avec son propre `pid` ; les threads d'un même processus partagent le même `tgid`.

4.3 Réalisation

Quatre fichiers du noyau ont été modifiés :

- `include/linux/sched.h` : ajout du champ `int is_vcpu` dans `task_struct` ;
- `arch/x86/entry/syscalls/syscall_64.tbl` : ajout du numéro **436** (les numéros 0 à 435 étant déjà utilisés en 5.4.1, le numéro 333 du sujet n'est plus libre) ;
- `include/linux/syscalls.h` : déclaration de `sys_make_vcpu` ;
- `kernel/sched/core.c` : définition de l'appel avec la macro `SYSCALL_DEFINE1`. Il retrouve le processus avec `find_process_by_pid`, parcourt ses threads avec `for_each_thread` sous protection RCU, positionne `is_vcpu` et trace chaque étape avec `printk` (préfixe `[VCPU]`). Il renvoie le nombre de threads marqués, ou `-ESRCH` si le processus n'existe pas.

L'ajout d'un champ à `task_struct` modifie la disposition de cette structure utilisée dans tout le noyau, qui a donc été recompilé entièrement, puis installé. La VM

démarre ensuite sur la version `#2` du noyau, et `System.map` contient bien la fonction `__x64_sys_make_vcpu`.

Le programme de test crée trois threads, puis appelle `syscall(436, getpid())`. L'appel renvoie 4 (le thread principal et les trois threads créés), et un PID inexistant donne l'erreur « No such process ». Le journal du noyau (`dmesg`) montre une ligne par thread : chacun a un `pid` différent, mais tous ont le même `tgid`.

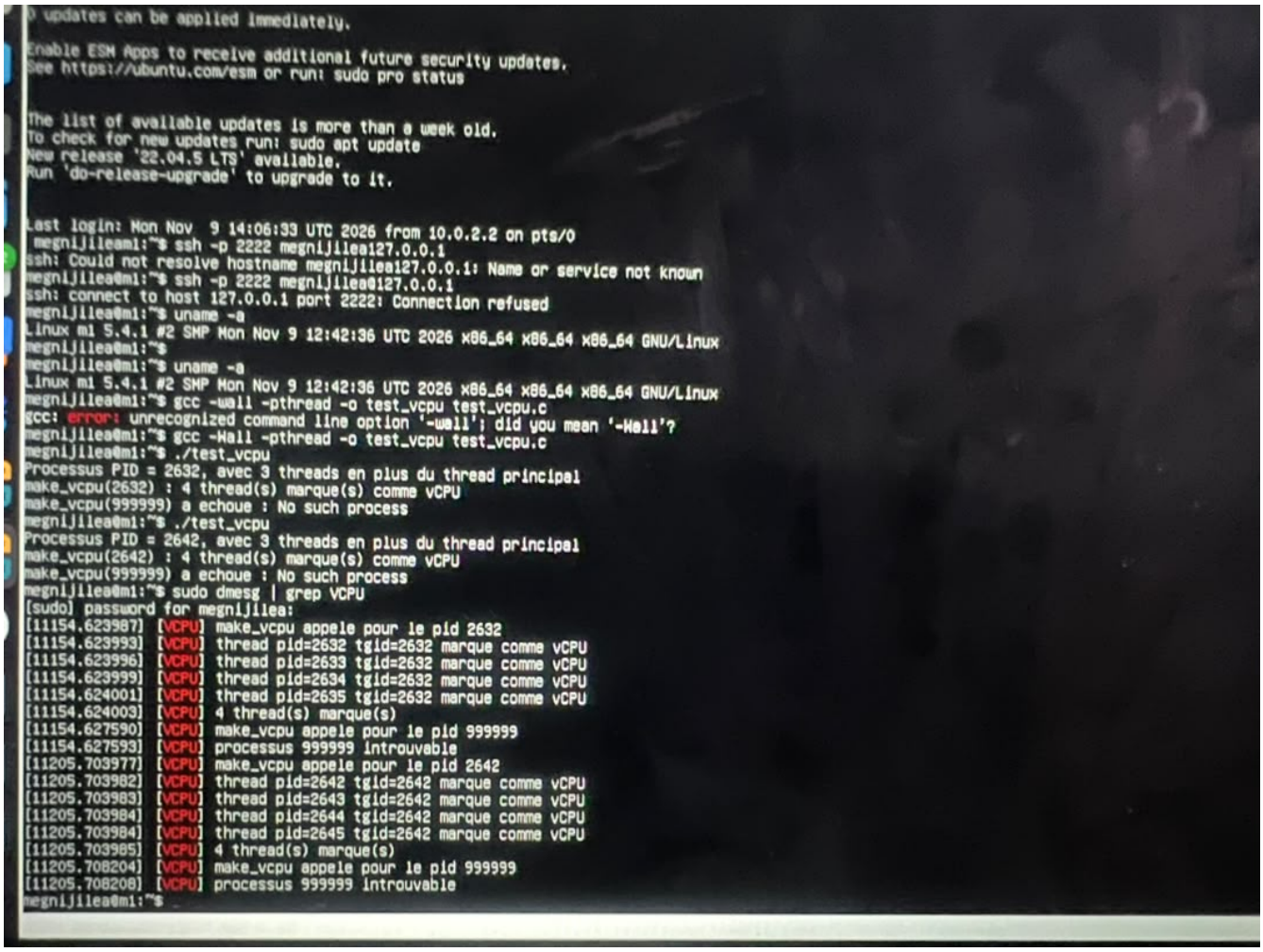


FIGURE 4.1 – Appel de make_vcpu et messages du noyau

4.4 Difficultés et conclusion

Lancé par erreur sur la machine hôte, le programme de test a obtenu « Invalid argument » : dans les noyaux récents, le numéro 436 correspond à l'appel système `close_range`. Cela montre qu'un numéro d'appel système n'a de sens que pour un noyau donné et doit être réservé officiellement. La recompilation complète, imposée par la modification de `task_struct`, a par ailleurs pris environ une heure et demie.

Ce TP montre comment le noyau expose ses services aux programmes, et qu'ajouter une fonctionnalité au noyau demande de modifier à la fois ses structures, sa table d'entrée et son code, puis de le recompiler.

Chapitre 5

TP5 – Gestionnaire de mémoire dynamique (*heap manager*)

5.1 Objectifs

Implémenter un gestionnaire de tas fournissant `malloc`, `free` et `realloc` (algorithme *first fit*, découpage et fusion des blocs), le tester, puis le faire utiliser par des programmes existants grâce à `LD_PRELOAD`.

5.2 Notions clés

- **Tas** : zone de mémoire où sont prises les allocations dynamiques. Le gestionnaire demande de grands morceaux de mémoire à l'OS et les redistribue.
- **`brk`/`sbrk` et `mmap`** : `sbrk` déplace la fin du tas (le *break*) ; `mmap` obtient une zone séparée, utilisée pour les grosses demandes (au-delà de 128 Ko) et rendue en entier par `munmap`.
- **Métadonnées** : chaque bloc est précédé d'un en-tête (taille, état, origine, pointeurs vers les blocs voisins) ; `free` retrouve cet en-tête à partir de l'adresse reçue.
- `LD_PRELOAD` : charge une bibliothèque avant la libc ; ses fonctions `malloc`, `free`… remplacent alors celles du système.

5.3 Réalisation

Les blocs, libres ou occupés, forment une liste doublement chaînée. Les tailles sont arrondies au multiple de 16 (alignement).

- `malloc` parcourt la liste et prend le premier bloc libre assez grand (*first fit*), qu'il découpe si le reste est suffisant pour former un nouveau bloc. À défaut, il demande de la mémoire à l'OS avec `sbrk`, ou avec `mmap` au-delà du seuil.
- `free` marque le bloc comme libre et le fusionne avec ses voisins libres et contigus en mémoire. Un bloc `mmap` est rendu par `munmap` ; un gros bloc libre situé en haut du tas est rendu à l'OS en reculant le *break*.
- `realloc` conserve le bloc s'il est assez grand, l'agrandit sur place si son voisin libre le permet, et sinon alloue un nouveau bloc, copie les données et libère l'ancien.
- `calloc` a été ajouté : sans lui, les programmes qui l'utilisent mélangeraient des blocs de la libc et des nôtres, ce qui provoquerait des plantages sous `LD_PRELOAD`.

Le programme de test affiche la liste des blocs après chaque opération : on y observe la réutilisation d'un trou libre et son découpage, la fusion des blocs libres voisins, la conservation du contenu après `realloc`, et l'utilisation de `mmap` pour une demande de 200 000 octets.

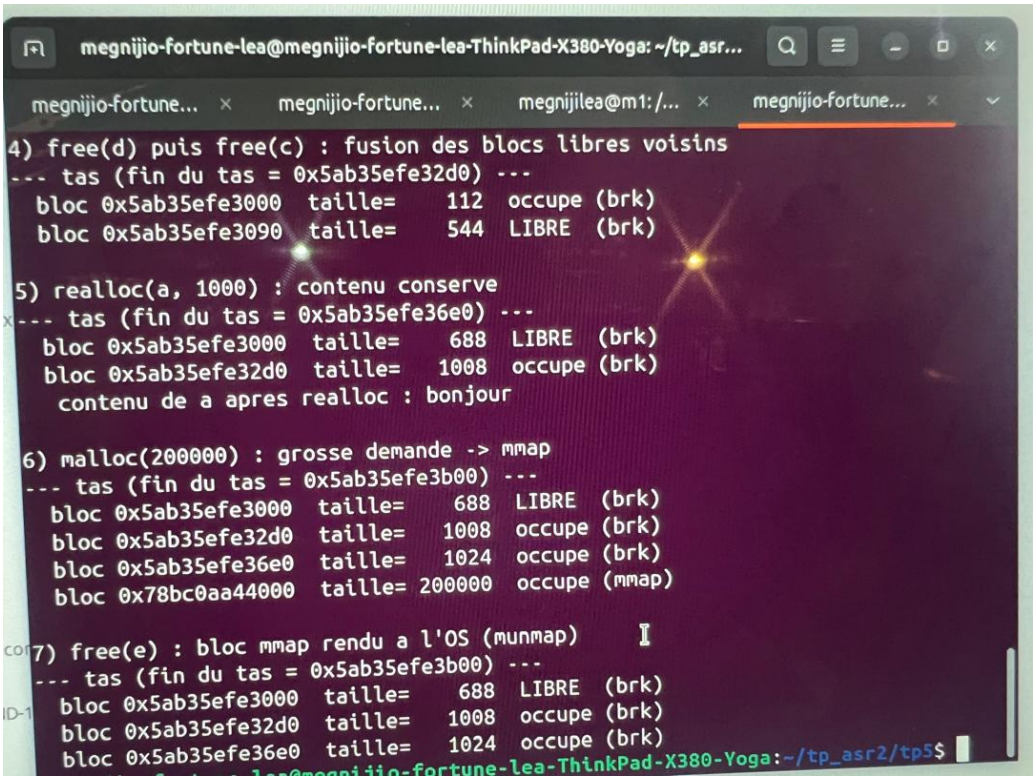


FIGURE 5.1 – Découpage, fusion, realloc et mmap observés sur la liste des blocs

Compilé en bibliothèque partagée (`gcc -shared -fPIC`), le gestionnaire est utilisé par `ls`, `date` et même l'interpréteur Python via `LD_PRELOAD`, qui fonctionnent normalement.

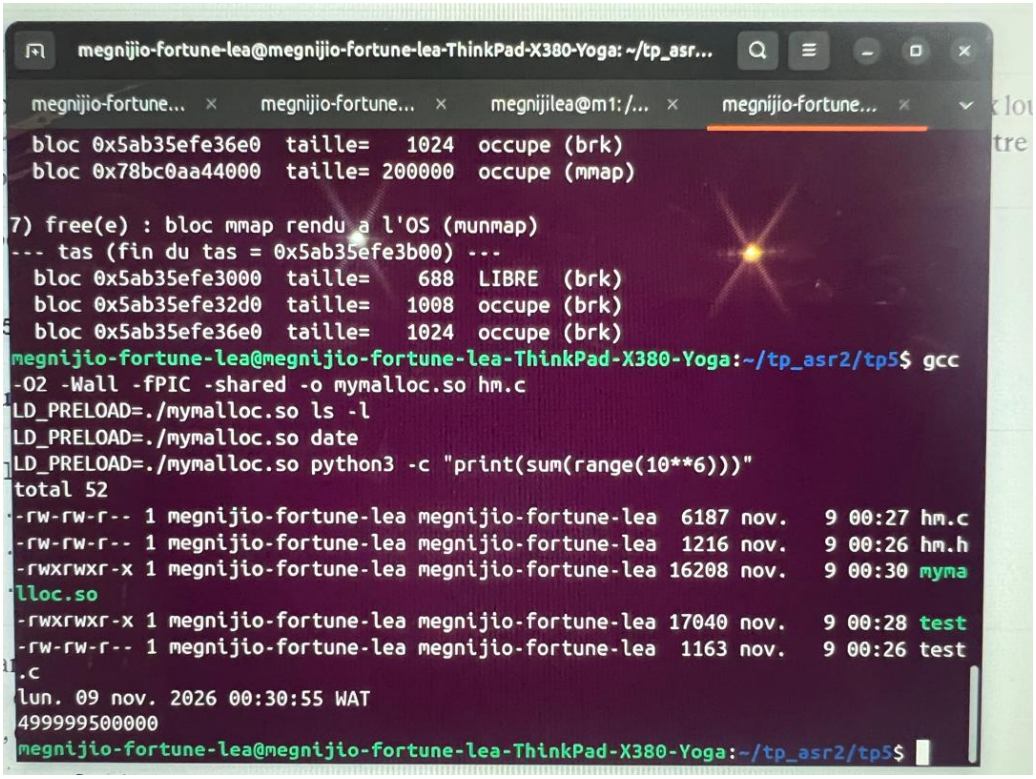


FIGURE 5.2 – Programmes existants utilisant notre gestionnaire via LD_PRELOAD

5.4 Difficultés et conclusion

Le gestionnaire ne doit jamais appeler `printf`, qui peut lui-même appeler `malloc` : l'affichage de test utilise donc `write`. Notre version n'est pas protégée contre les accès simultanés de plusieurs threads, ce qui suffit pour les programmes testés.

Ce TP montre que `malloc` n'est pas un appel système mais une bibliothèque qui gère elle-même la mémoire obtenue de l'OS, en cherchant un compromis entre rapidité et fragmentation. Les fichiers sources sont remis séparément.

Chapitre 6

TP6 – Protection mémoire par pages de garde

6.1 Objectifs

Enrichir le gestionnaire de mémoire du TP5 d'une allocation protégée, `pmalloc`, qui détecte les débordements de tampon (*buffer overflow*) grâce à une page de garde placée juste après chaque buffer, rendue inaccessible par `mprotect` puis, dans une seconde version, retirée par `munmap`.

6.2 Notions clés

- **Buffer overflow** : écriture au-delà de la taille d'un buffer, qui écrase silencieusement les données voisines ; c'est une faille de sécurité classique.
- **Page** : unité de gestion de la mémoire (4 096 octets, obtenue avec `sysconf(_SC_PAGE_SIZE)`). Les droits d'accès sont attribués par page et vérifiés par le processeur à chaque accès.
- **Page de garde** : page sans aucun droit (`PROT_NONE`) ou absente de l'espace d'adresses ; tout accès provoque le signal `SIGSEGV`.

6.3 Réalisation

`pmalloc` obtient par `mmap` les pages nécessaires au buffer et à son en-tête, plus une page de garde. Le buffer est placé **à la fin** des pages de données, contre la page de garde : le premier octet en trop touche ainsi la page interdite. La page de garde est protégée soit avec `mprotect(..., PROT_NONE)`, soit retirée avec `munmap`. `pfree` rend ses droits à la page de garde (version `mprotect`) puis libère l'ensemble.

Un gestionnaire de `SIGSEGV`, installé avec `sigaction` et l'option `SA_SIGINFO`, reçoit l'adresse fautive : si elle appartient à une page de garde, il signale une tentative de buffer overflow, puis termine le programme.

Le test montre d'abord qu'avec un `malloc` classique, un débordement de 48 octets écrase un mot de passe stocké dans le buffer voisin sans aucune erreur. Avec `pmalloc`, une écriture normale et une réallocation après `pfree` fonctionnent, tandis que le débordement est arrêté dès le premier octet en trop, avec les deux méthodes.

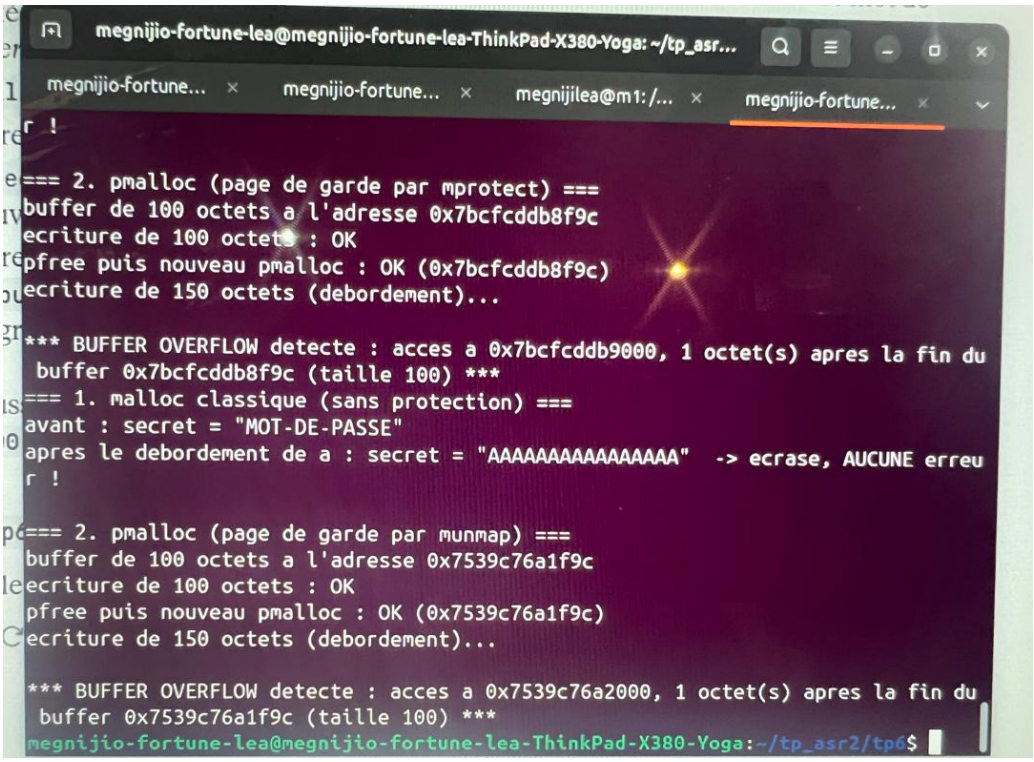


FIGURE 6.1 – Débordement silencieux avec malloc, puis détecté avec pmalloc

6.4 Difficultés et conclusion

Placer le buffer exactement contre la page de garde empêche de l'aligner sur 16 octets : nous avons privilégié la détection immédiate. La méthode `munmap` est moins sûre que `mprotect`, car l'adresse libérée peut être réutilisée par une autre allocation, ce qui supprimerait la protection.

La page de garde offre une protection efficace et gratuite à l'exécution, puisqu'elle repose sur le matériel. Elle coûte en revanche au moins deux pages par allocation : elle est donc réservée aux buffers critiques ou au débogage. Les fichiers sources sont remis séparément.

Chapitre 7

TP7 – Suivi des pages *soft-dirty* d'un processus

7.1 Objectifs

Écrire un programme qui, pour un processus donné, parcourt sa table des pages et affiche les pages modifiées (partie 1), puis remet périodiquement ces indicateurs à zéro pour suivre les pages modifiées entre deux mesures, jusqu'à ce que leur nombre devienne presque constant (partie 2).

7.2 Notions clés

- **Table des pages** : associe chaque page virtuelle d'un processus à une page physique ; le noyau y conserve aussi des indicateurs par page.
- **Bit *soft-dirty*** : passe à 1 lorsqu'une page est écrite. Il est lisible, pour chaque page, dans `/proc/PID/pagemap` (enregistrement de 64 bits par page, bit 55).
- `/proc/PID/maps` : liste des zones réellement utilisées par le processus ; `/proc/PID/clear_refs` : écrire `4` remet tous les bits *soft-dirty* à zéro.
- **Usage** : un outil de migration comme CRIU recopie la mémoire d'un processus en cours d'exécution, puis seulement les pages modifiées entre-temps, jusqu'à ce que leur nombre se stabilise.

7.3 Réalisation

Le programme `suivi` lit `/proc/PID/maps` et ne parcourt que les zones accessibles en écriture. Pour chaque page d'adresse `adr`, il lit avec `pread` l'enregistrement situé à la position (`adr`/4096) $\times$ 8 de `pagemap` et teste le bit 55. Un processus `cible` sert de sujet d'observation : il modifie 1 000 pages au démarrage, de moins en moins pendant 10 secondes, puis 50 pages en continu.

Partie 1. Le programme affiche les adresses des pages sales par zone (tas, pile, bibliothèques…) : 1 060 pages au total, dont les 1 000 pages de la zone allouée par la cible.

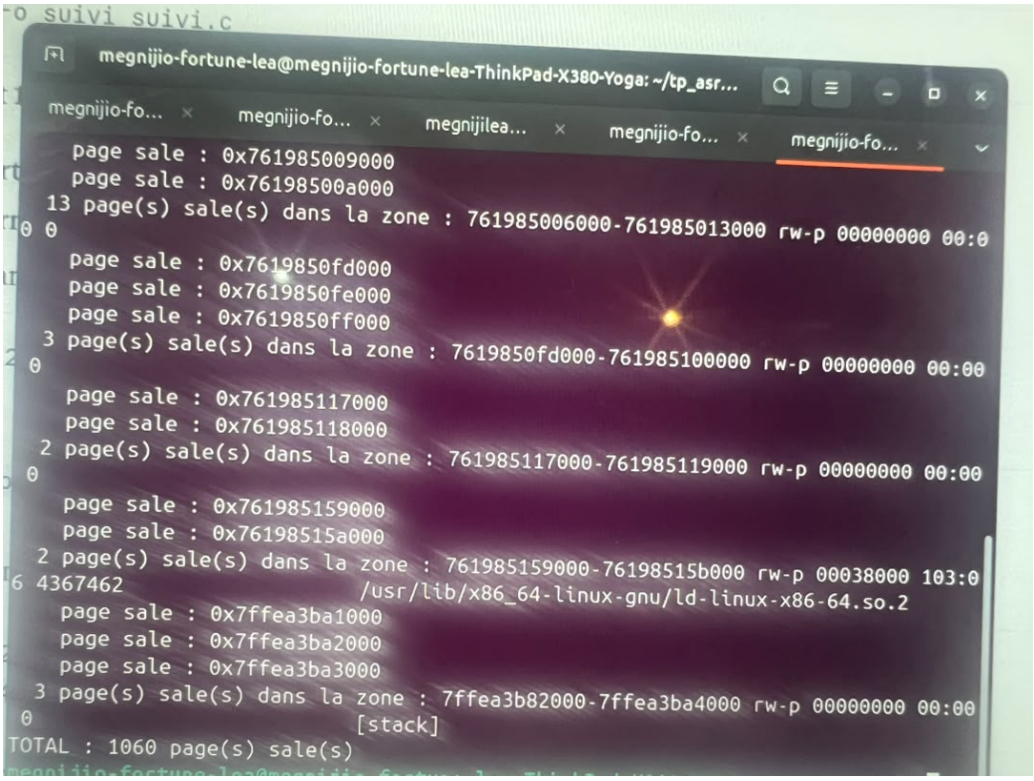


FIGURE 7.1 – Pages soft-dirty du processus cible, par zone

Partie 2. À chaque tour, le programme écrit `4` dans `clear_refs`, attend une seconde, puis compte les pages modifiées pendant cet intervalle. Il s'arrête lorsque le nombre varie de moins de 5 % pendant trois tours consécutifs. Une fois la phase initiale de la cible terminée, on mesure 52 pages modifiées par seconde : les 50 pages actives de la cible, plus sa pile et ses variables.

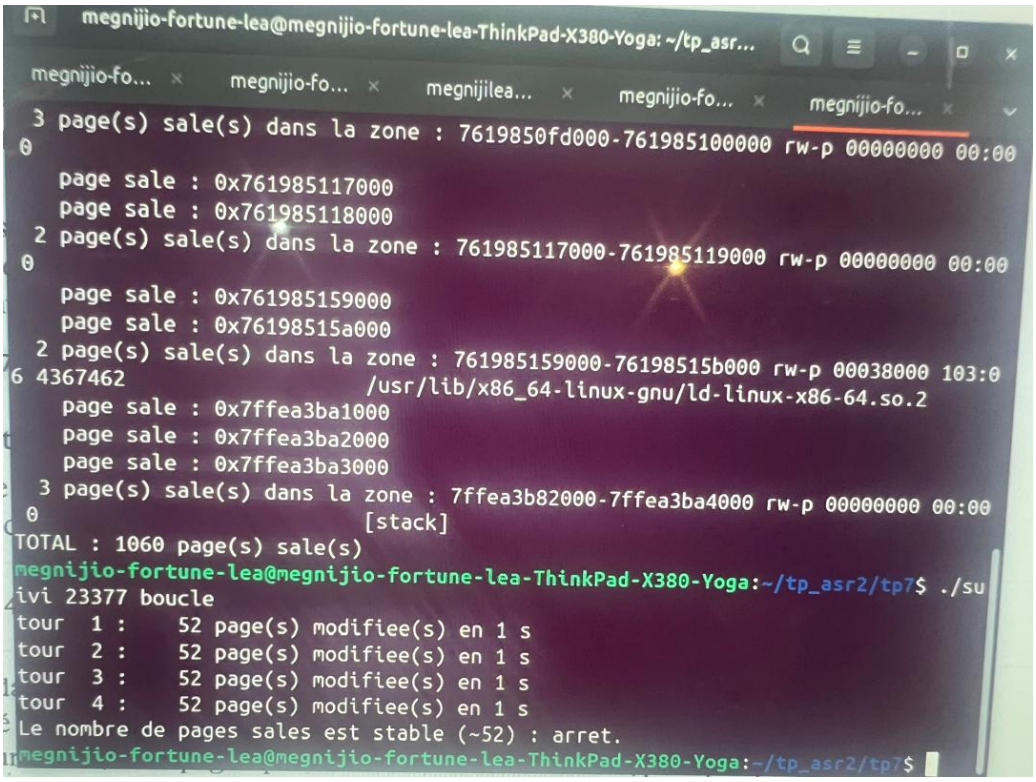


FIGURE 7.2 – Suivi périodique jusqu'à stabilisation du nombre de pages sales

7.4 Difficultés et conclusion

Parcourir tout l'espace virtuel (128 To) serait beaucoup trop long : se limiter aux zones de `/proc/PID/maps`, et seulement à celles accessibles en écriture, rend le parcours instantané. Le noyau doit être compilé avec l'option `CONFIG_MEM_SOFT_DIRTY`, active sur Ubuntu.

Ce TP montre qu'un processus peut observer la mémoire d'un autre grâce aux informations exposées par le noyau dans `/proc`, ce qui est à la base des techniques de sauvegarde et de migration à chaud. Les fichiers sources sont remis séparément.

Chapitre 8

TP8 – Module noyau et pilote en espace utilisateur (UIO)

8.1 Objectifs

Écrire un module noyau qui crée un périphérique UIO (*Userspace I/O*) partageant une page de mémoire avec un programme utilisateur, et qui suit, grâce aux notifications de préemption, les passages sur processeur de processus dont les PID sont donnés en paramètre. Le programme utilisateur est réveillé par des interruptions simulées et affiche les informations reçues.

8.2 Notions clés

- **Module noyau** : code chargé (`insmod`) et déchargé (`rmmod`) à chaud, sans recompiler le noyau ; ses fonctions `module_init` et `module_exit` préparent puis défont toutes ses ressources.
- **UIO** : seul le minimum du pilote est dans le noyau (déclaration du périphérique, accès à sa mémoire, interruptions) ; le reste est un programme ordinaire, qui ouvre `/dev/uio0`, projette la mémoire du périphérique avec `mmap` et attend les interruptions avec `read`.
- **Notifications de préemption** : fonctions `sched_in` et `sched_out` appelées par l'ordonnanceur lorsqu'un processus est placé sur un processeur ou en est retiré (mécanisme utilisé par KVM pour ses processeurs virtuels).

8.3 Réalisation

Au chargement, le module enregistre un périphérique virtuel (`platform_device_register_simple`), alloue une page partagée et la décrit dans une structure `uio_info`, puis l'enregistre avec `uio_register_device`, ce qui crée `/dev/uio0`. Pour chaque PID reçu (`module_param_array`), il retrouve la tâche (`pid_task(find_vpid(pid))`) et y attache un notifieur de préemption. Les fonctions de notification se contentent d'incrémenter, dans la page partagée, les compteurs d'entrées et de sorties du processus et de noter son processeur. Une minuterie appelle chaque seconde `uio_event_notify`, qui simule une interruption et réveille le programme utilisateur.

Trois adaptations ont été faites par rapport au squelette fourni : `find_task_by_vpid`, non exportée, est remplacée par des fonctions exportées, ce qui évite de recompiler le noyau ; `preempt_notifier_register` ne s'appliquant qu'au processus courant, le notifieur est ajouté directement à la liste `preempt_notifiers` de la tâche surveillée ; enfin, les interruptions sont envoyées par la minuterie et non depuis les notifications, appelées à un moment où l'ordonnanceur ne doit pas être bloqué.

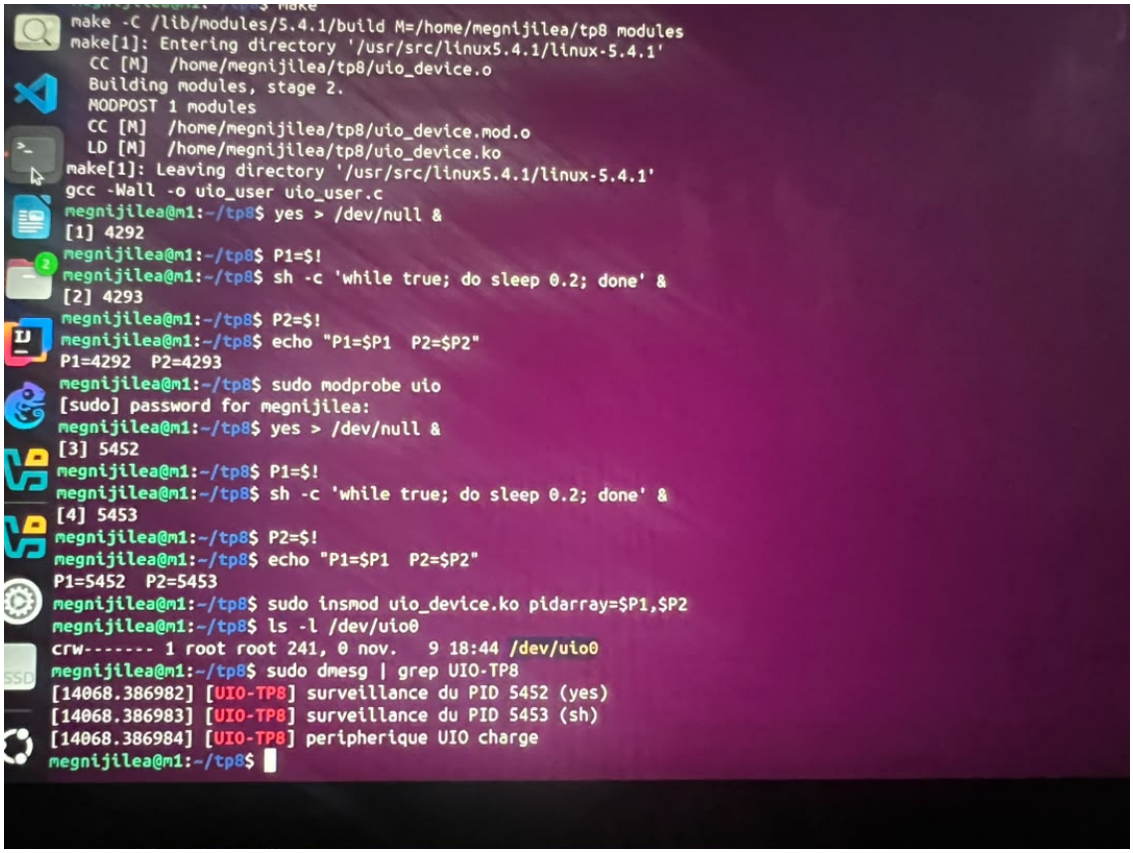


FIGURE 8.1 – Chargement du module, périphérique /dev/uio0 et messages du noyau

Le test surveille deux processus : `yes`, qui calcule en permanence, et une boucle qui dort 0,2 s entre deux réveils. Après environ 400 interruptions, le premier ne compte que 306 passages sur processeur, contre 2 063 pour le second : un processus qui dort quitte volontairement le processeur à chaque attente (5 fois par seconde ici), alors qu'un processus de calcul n'en est retiré que lorsqu'un autre processus a besoin du processeur. Les entrées et les sorties sont toujours égales.

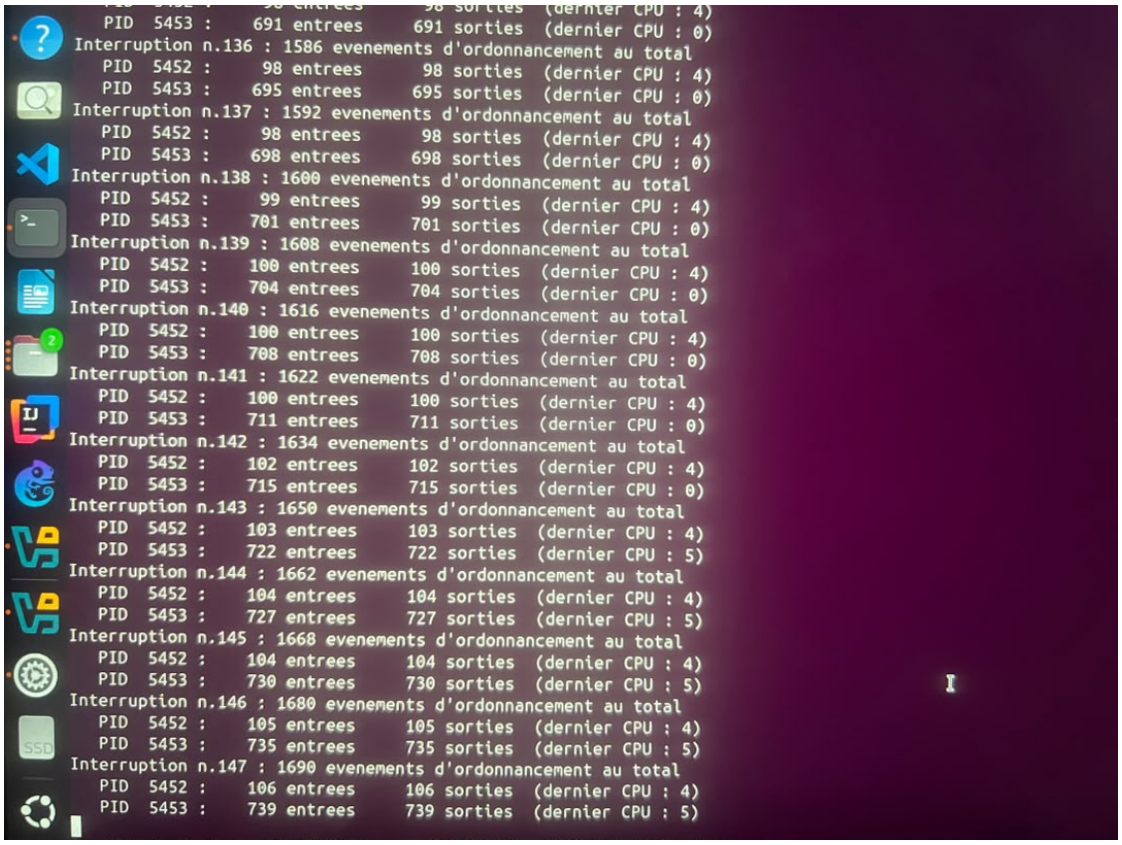


FIGURE 8.2 – Compteurs d'ordonnancement reçus par le programme utilisateur

8.4 Difficultés et conclusion

Un `Makefile` exige des tabulations, perdues lors du copier-coller dans le terminal : nous avons utilisé `.RECIPEPREFIX` et placé la ligne `obj-m` dans un fichier `Kbuild`. Par ailleurs, les processus surveillés doivent être arrêtés avant de décharger le module, pour ne pas retirer un notifieur pendant que l'ordonnanceur le parcourt.

Ce TP montre comment étendre le noyau à chaud par un module, comment UIO permet d'écrire l'essentiel d'un pilote en espace utilisateur, et comment observer les décisions de l'ordonnanceur. Les fichiers sources sont remis séparément.

Chapitre 9

TP9 – Communication entre processus : `pipe` et `dup2`

9.1 Objectifs

Programmer le crible d'Ératosthène avec deux processus qui communiquent par un tube (`pipe`), et rediriger le résultat dans un fichier avec `dup2`.

9.2 Notions clés

- **Descripteur de fichier** : numéro par lequel un processus désigne un fichier ouvert ; 0, 1 et 2 sont l'entrée standard, la sortie standard et la sortie d'erreur.
- **Tube (`pipe`)** : canal à sens unique fourni par le noyau, avec une extrémité d'écriture et une extrémité de lecture. Créé avant un `fork`, il est hérité par le fils et relie les deux processus. Le lecteur reçoit la fin de fichier lorsque tous les descripteurs d'écriture sont fermés.
- **`dup2(a, b)`** : fait pointer le descripteur `b` vers le même fichier que `a`. Avec `b = 1`, tout ce que le processus affiche part dans le fichier : c'est la redirection `>` du shell.
- **Crible d'Ératosthène** : on raye les multiples de chaque nombre premier à partir de son carré ; les nombres restants sont premiers.

9.3 Réalisation

Le père crée le tube, puis le fils. Chaque processus ferme l'extrémité qu'il n'utilise pas.

- **Le père (générateur)** écrit les entiers de 2 à N dans le tube, puis ferme son extrémité d'écriture et attend le fils.
- **Le fils (crible)** redirige sa sortie standard vers `premiers.txt` avec `dup2`, lit les nombres jusqu'à la fin du tube, applique le crible, puis affiche les nombres premiers avec un simple `printf`, qui écrit donc dans le fichier.

Le résultat est correct : 25 nombres premiers jusqu'à 100, et 78 498 jusqu'à un million, la valeur connue.

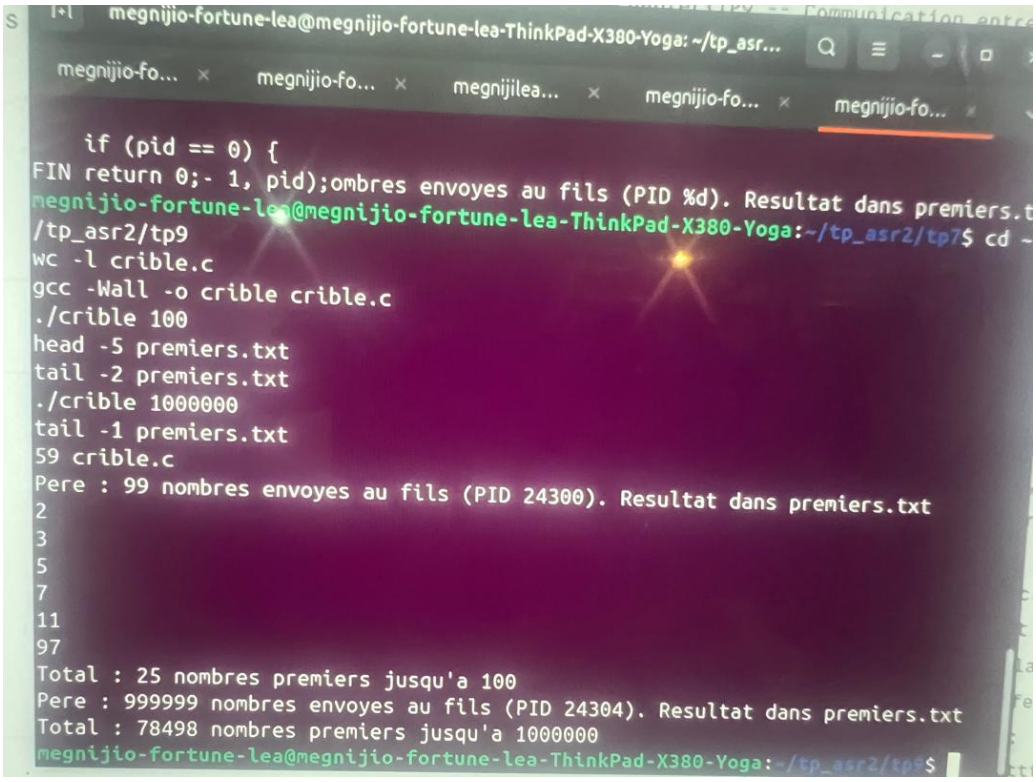


FIGURE 9.1 – Exécution du crible et contenu du fichier premiers.txt

9.4 Difficultés et conclusion

La principale difficulté est la fermeture des extrémités inutilisées du tube : si le fils gardait son descripteur d'écriture ouvert, il n'atteindrait jamais la fin du tube et resterait bloqué en lecture.

Ce TP illustre deux mécanismes fondamentaux du shell : le tube, qui relie deux commandes (`commande1 | commande2`), et la redirection de la sortie standard (`commande > fichier`). Le fichier source est remis séparément.

Conclusion générale

Ces travaux pratiques nous ont permis de passer de l'utilisation de Linux à la compréhension de son fonctionnement. Plusieurs idées traversent l'ensemble des TP.

La frontière entre l'espace utilisateur et le noyau. Les programmes n'accèdent jamais directement au matériel : ils passent par des appels système. Nous l'avons vu de l'extérieur, en utilisant `fork`, `exec`, `pipe` ou `mmap`, puis de l'intérieur, en compilant le noyau et en lui ajoutant notre propre appel système.

Les processus et leur contrôle. Le mini-shell, les threads et le crible d'Ératosthène montrent comment des processus sont créés, communiquent et sont pilotés par des signaux, et comment un ordonnanceur peut être réalisé par le noyau ou par le programme lui-même grâce aux contextes.

La mémoire comme ressource gérée en couches. Le matériel découpe la mémoire en pages et contrôle leurs droits ; le noyau associe pages virtuelles et physiques, et expose leur état dans `/proc` ; les bibliothèques comme `malloc` redistribuent cette mémoire aux programmes. Le gestionnaire de tas, les pages de garde et le suivi des pages *soft-dirty* nous ont fait intervenir à chacun de ces niveaux.

Sur le plan pratique, ces TP nous ont aussi appris à travailler avec méthode : lire la documentation officielle (pages de manuel, documentation du noyau), vérifier chaque étape avant de passer à la suivante, sauvegarder avant toute opération risquée, et adapter un sujet à un environnement différent de celui prévu. Les difficultés rencontrées, comme l'espace disque insuffisant, une installation interrompue, la syntaxe stricte de `/etc/fstab` ou les interactions entre signaux et groupes de processus, ont été autant d'occasions de comprendre plus finement le système.

Ces connaissances constituent une base solide pour aborder des sujets plus avancés, comme la virtualisation, la sécurité des systèmes ou le développement de pilotes de périphériques.

Bibliographie

[1] A. Tchana, S. Bitchebe, L. Wapet, *Sujets des TP1 à TP9 du cours ASR2*, ENS Lyon, 2019–2021.

[2] The Linux Kernel Archives, `https://www.kernel.org/`

[3] The Linux Kernel documentation, `https://docs.kernel.org/`

[4] Linux Kernel documentation, *Adding a New System Call*, `https://docs.kernel.org/process/adding-syscalls.html`

[5] Linux Kernel documentation, *Kconfig* et *Kbuild*, `https://docs.kernel.org/kbuild/`

[6] Bootlin, *Elixir Cross Referencer – Linux 5.4.1*, `https://elixir.bootlin.com/linux/v5.4.1/source`

[7] M. Kerrisk, *The Linux man-pages project*, `https://man7.org/linux/man-pages/` – notamment `fork(2)`, `execve(2)`, `wait(2)`, `signal(7)`, `sigaction(2)`, `pthreads(7)`, `makecontext(3)`,

[8] J. Huang, *Malloc (parties 1 et 2)*, University of Tennessee, `http://web.eecs.utk.edu/~huangj/cs360/360/notes/Malloc1/lecture.html`

[9] Intersec, *Memory – Part 3 : Managing memory*, `https://techtalk.intersec.com/2013/08/memory-part-3-managing-memory/`

[10] Linux Kernel documentation, *Soft-Dirty PTEs* et *Examining Process Page Tables (pagemap)*, `https://docs.kernel.org/admin-guide/mm/soft-dirty.html`

[11] LWN.net, *Soft-dirty bit*, `https://lwn.net/Articles/546966/`

[12] Oracle, *VirtualBox User Guide*, `https://www.virtualbox.org/manual/`

[13] Canonical, *Ubuntu Server documentation*, `https://ubuntu.com/server/docs`

[14] GNU GRUB Manual, `https://www.gnu.org/software/grub/manual/grub/grub.html`

[15] Wikipédia, *Master boot record*, `https://fr.wikipedia.org/wiki/Master_boot_record`

[16] Wikipédia, *Crible d'Ératosthène*, `https://fr.wikipedia.org/wiki/Crible_d%27%C3%89ratosth%C3%A8ne`

Annexe A

Codes sources

Les codes sources des travaux pratiques n'ont pas été reproduits dans ce rapport. Ce choix est volontaire : le document est déjà conséquent, et l'ajout de plusieurs centaines de lignes de code l'aurait fortement alourdi sans apporter d'information supplémentaire à la compréhension des TP. Le rapport se concentre donc sur les notions, la démarche et les résultats obtenus, illustrés par des captures d'écran.

L'ensemble des fichiers sources est fourni séparément, sous forme de fichiers récupérables sur un support amovible (clé USB, disque dur externe, etc.). Ils sont organisés en un dossier par TP :

- **TP2** : `minishell.h`, `minishell.c` ;
- **TP3** : `threads.c`, `contextes.c` ;
- **TP4** : les modifications apportées au noyau (`include/linux/sched.h`, `include/linux/syscall`
  `arch/x86/entry/syscalls/syscall_64.tbl`, `kernel/sched/core.c`) et le programme de test ;
- **TP5** : `hm.h`, `hm.c`, `test.c` ;
- **TP6** : `pmalloc.h`, `pmalloc.c`, `test_garde.c` ;
- **TP7** : `cible.c`, `suivi.c` ;
- **TP8** : `uio_device.c`, `uio_user.c`, `Makefile` ;
- **TP9** : `crible.c`.

Chaque programme se compile avec `gcc` selon les commandes indiquées dans le chapitre correspondant du rapport.

Annexe A

Codes sources

Les codes sources des travaux pratiques n'ont pas été reproduits dans ce rapport. Ce choix est volontaire : le document est déjà conséquent, et l'ajout de plusieurs centaines de lignes de code l'aurait fortement alourdi . Le rapport se concentre donc sur les notions, la démarche et les résultats obtenus, illustrés par des captures d'écran.

L'ensemble des fichiers sources est fourni séparément, sous forme de fichiers récupérables sur un support amovible (clé USB, disque dur externe, etc.). Ils sont organisés en un dossier par TP :

- **TP2** : `minishell.h`, `minishell.c` ;
- **TP3** : `threads.c`, `contextes.c` ;
- **TP4** : les modifications apportées au noyau (`include/linux/sched.h`, `include/linux/syscalls`
  `arch/x86/entry/syscalls/syscall_64.tbl`, `kernel/sched/core.c`) et le programme de test ;
- **TP5** : `hm.h`, `hm.c`, `test.c` ;
- **TP6** : `pmalloc.h`, `pmalloc.c`, `test_garde.c` ;
- **TP7** : `cible.c`, `suivi.c` ;
- **TP8** : `uio_device.c`, `uio_user.c`, `Makefile` ;
- **TP9** : `crible.c`.

Chaque programme se compile avec `gcc` selon les commandes indiquées dans le chapitre correspondant du rapport.